

Time shared architecture

Ch-9: Book: Dr Shoab

Ch-8,10: Book: Stephen Brown

Architecture design

- Nyquist Sampling Theorem $f_s \geq 2f_N$
- Fully dedicated architecture
 - Unfolded architecture
- Time shared architecture
 - Folded architecture
 - Bit serial architecture
 - Digit serial architecture

Fully dedicated architecture

Example

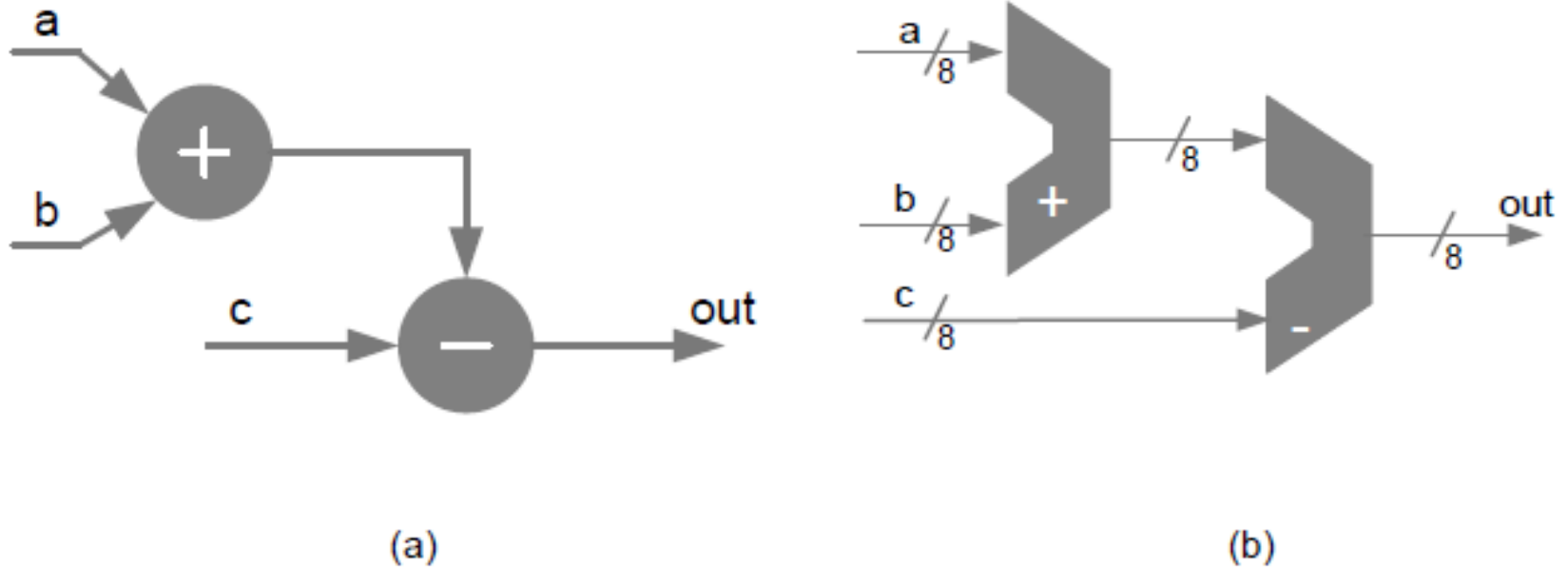
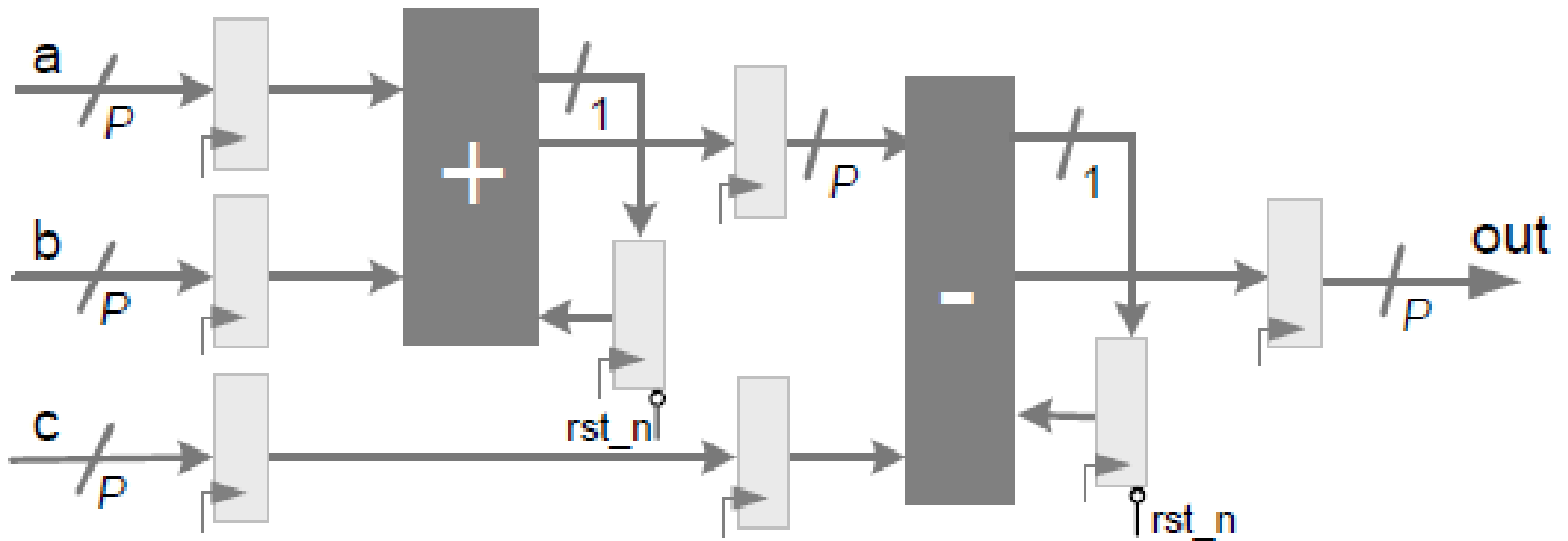
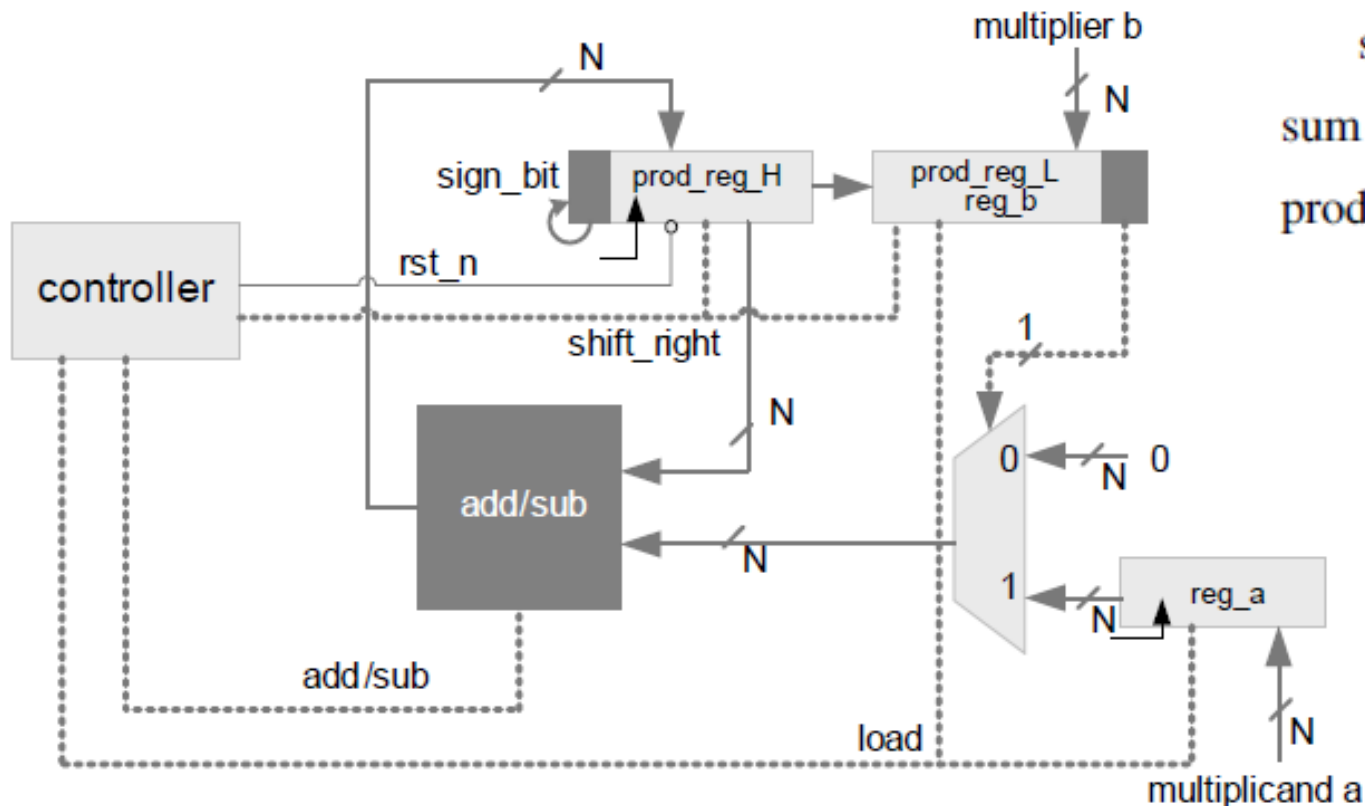


Figure 9.1 (a) Algorithm requiring equal sampling and clock frequencies. (b) Mapping on FDA

Bit serial architecture



Digit serial architecture



$sum = 0$

for($i = 0; i < N-1; i++$)

$sum+ = a \times b[i] \times 2^i$

$sum- = a \times b[N-1] \times 2^{N-1}$

prod = sum

Figure 9.3 An $N \times N$ bit sequential multiplier

Sequential circuit

- Outputs depend on the past behavior of the circuit, as well as on the present values of inputs
- Also known as Finite state machines (FSM)
- Synchronous Sequential circuit
 - a clock signal is used to control the operation of the sequential circuit
- Asynchronous Sequential circuit
 - no clock signal is used

Sequential circuit realization

- Inputs: W
- Outputs: Z
- States: Q
- Types: Mealy and Moore

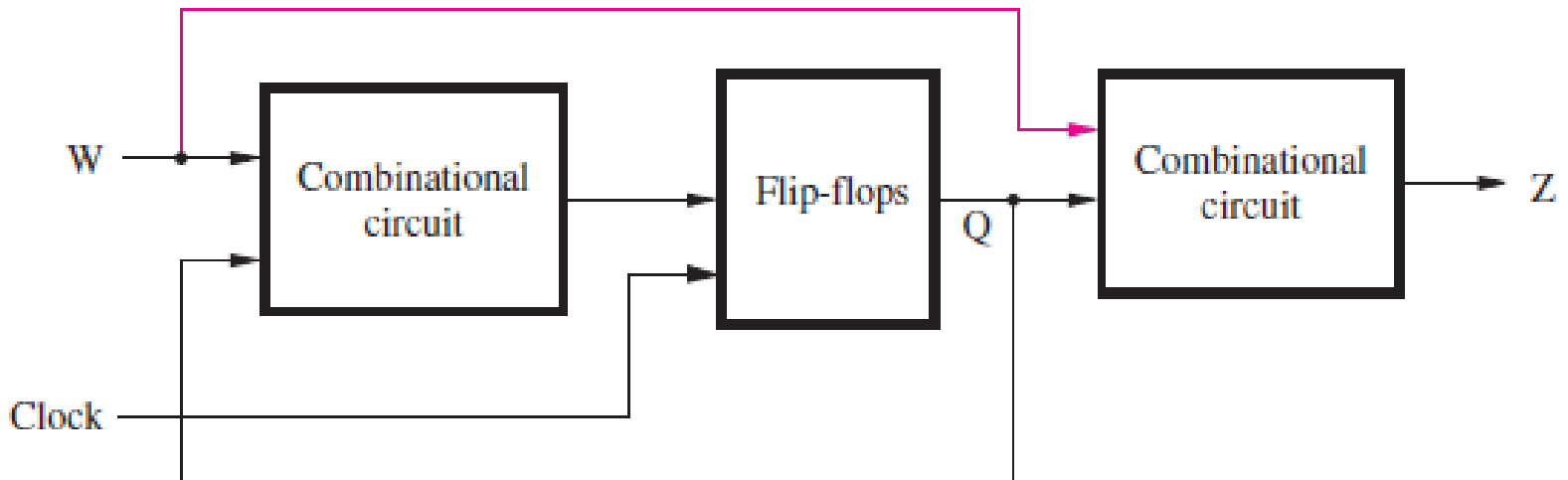


Figure 8.1 The general form of a sequential circuit.

FSM implementation

- Two general implementation methods
 - Hardwired
 - Micro-programmed
- Two representation
 - State Diagram
 - Algorithmic State Machine
- Two Types of State Machines
 - Mealy State Machine
 - Moor State Machine

Design example- Moore

- 1. The circuit has one input, w , and one output, z .
- 2. All changes in the circuit occur on the positive edge of a clock signal.
- 3. The output z is equal to 1 if during two immediately preceding clock cycles the input w was equal to 1. Otherwise, the value of z is equal to 0.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

State machine design

- Let the starting state is called state 'A'. *As long as the input w is 0*
- When w becomes equal to 1, the machine move to a different state 'B', keeping output $z=0$
- When in state B, if w is 0 at the next active clock edge, the circuit should move back to state A. However, if $w = 1$ when in state B, the circuit should change to a third state, C, and it should then generate an output $z = 1$
- The circuit should remain in state 'C' as long as $w = 1$ and should continue to maintain $z = 1$.
- When w becomes 0, the machine should move back to state A

State diagram

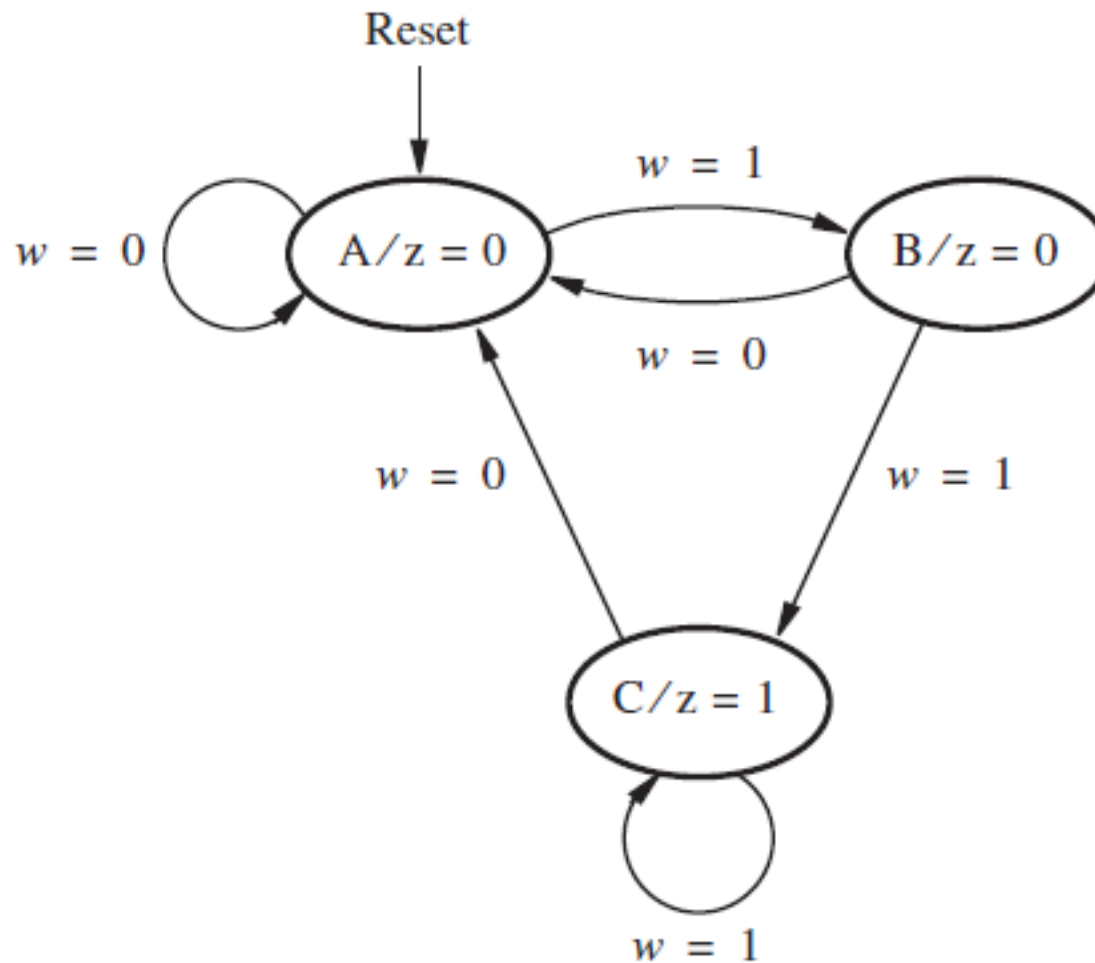


Figure 8.3 State diagram of a simple sequential circuit.

State table

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

Figure 8.4 State table for the sequential circuit in Figure 8.3.

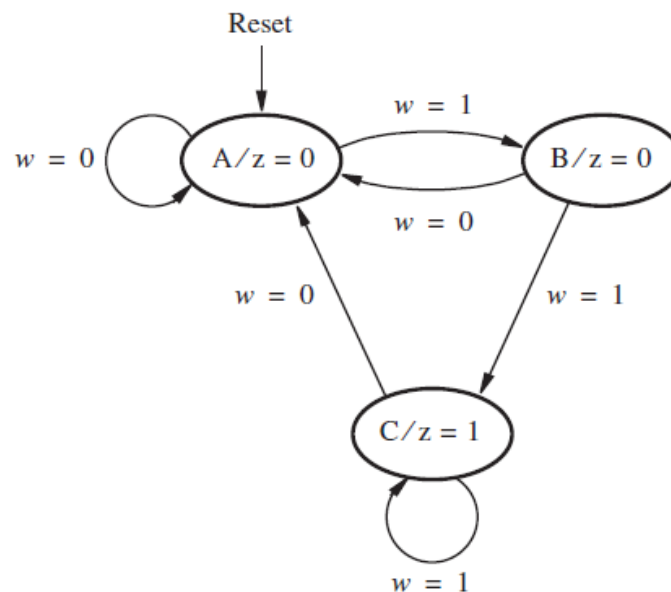


Figure 8.3 State diagram of a simple sequential circuit.

Generalized circuit

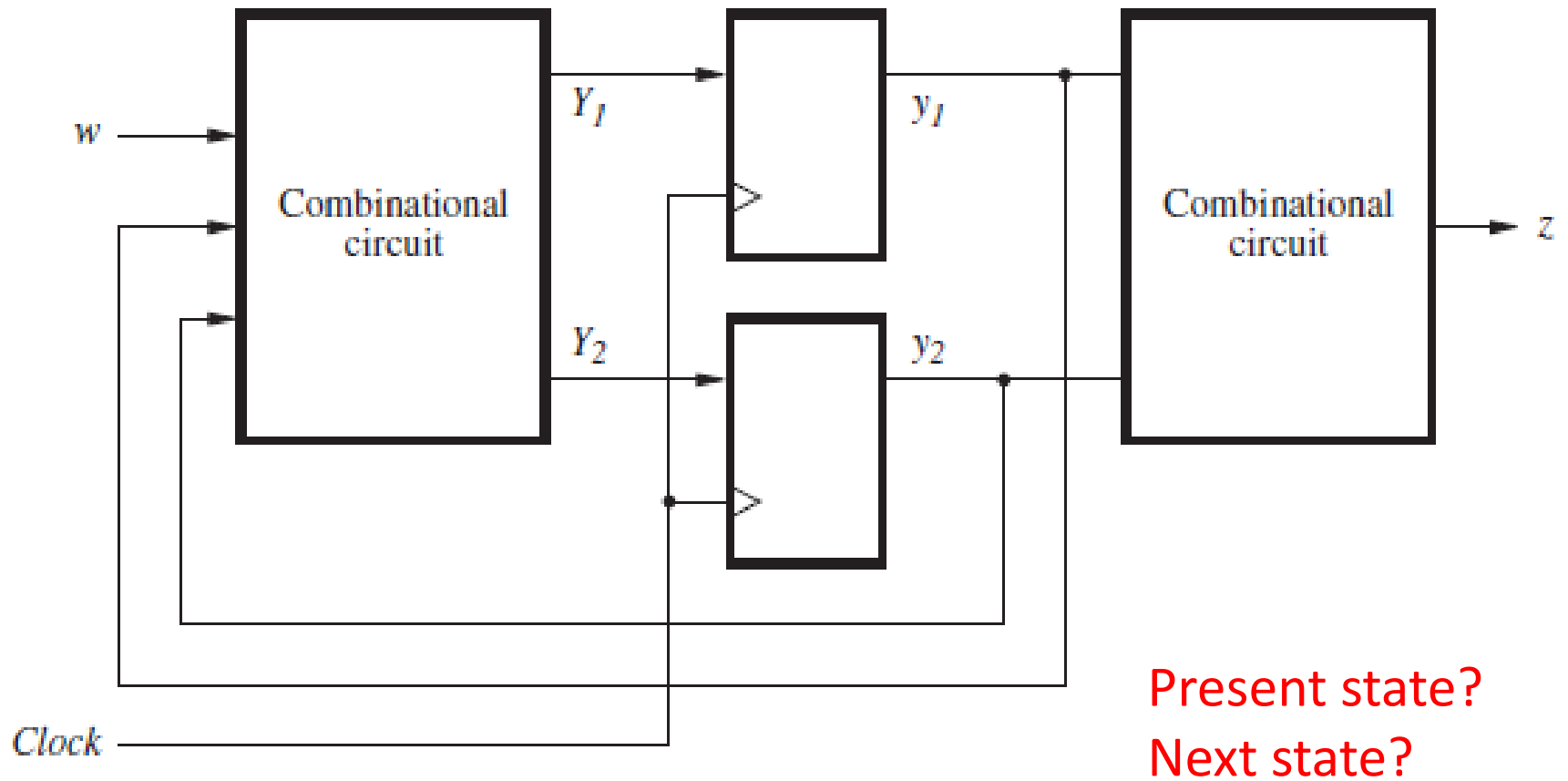


Figure 8.5 A general sequential circuit with input w , output z , and two state flip-flops.

State assignment

	Present state y_2y_1	Next state		Output z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Figure 8.6 State-assigned table for the sequential circuit in Figure 8.4.

Combinational circuit

$y_2 y_1$		00	01	11	10
w	0	0	0	d	0
	1	1	0	d	0

Ignoring don't cares

$$Y_1 = w\bar{y}_1\bar{y}_2$$

Using don't cares

$$Y_1 = w\bar{y}_1\bar{y}_2$$

$y_2 y_1$		00	01	11	10
w	0	0	0	d	0
	1	0	1	d	1

$$Y_2 = wy_1\bar{y}_2 + w\bar{y}_1y_2$$

$$\begin{aligned} Y_2 &= wy_1 + wy_2 \\ &= w(y_1 + y_2) \end{aligned}$$

y_2	y_1		0	1
	0	0	0	0
1	1	1	d	

$$z = \bar{y}_1y_2$$

$$z = y_2$$

Figure 8.7 Derivation of logic expressions for the sequential circuit in Figure 8.6.

Final implementation

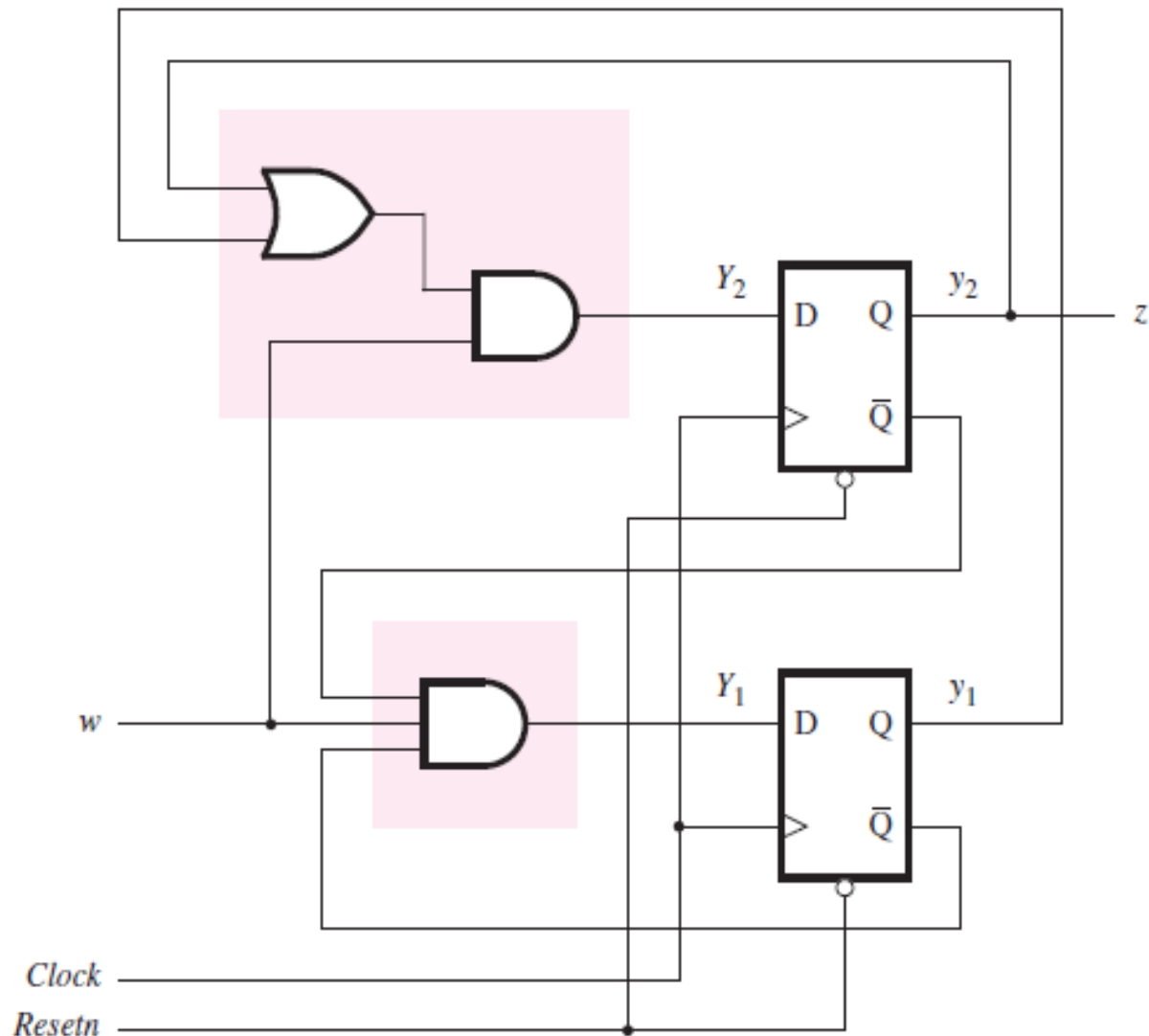


Figure 8.8 Final implementation of the sequential circuit in Figure 8.7.

Timing diagram

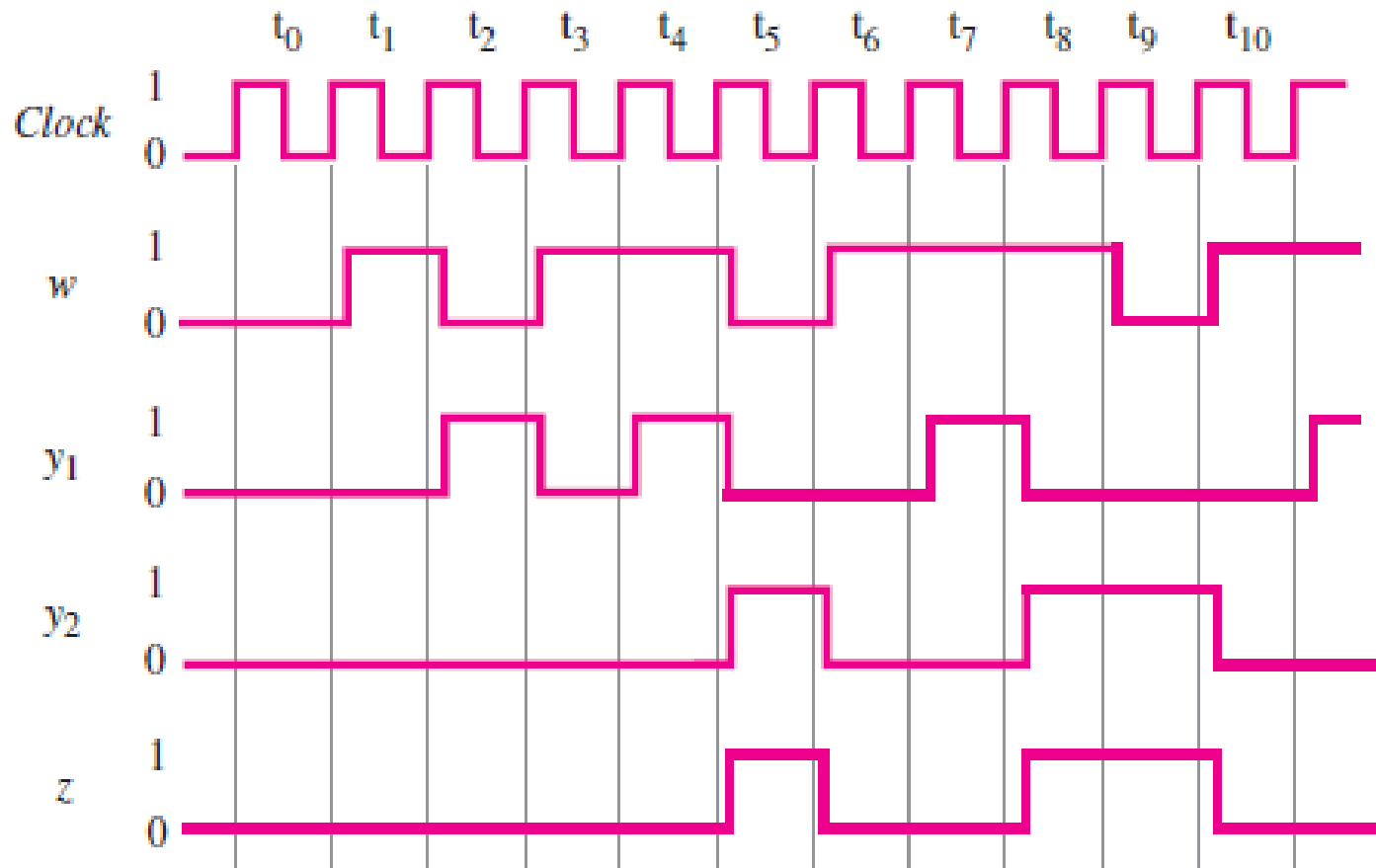


Figure 8.9 Timing diagram for the circuit in Figure 8.8.

- Verilog code
Moore
FSM

- Verilog code
Moore
FSM, second
version

- Verilog code
Moore
FSM, third
version

Mealy State Model

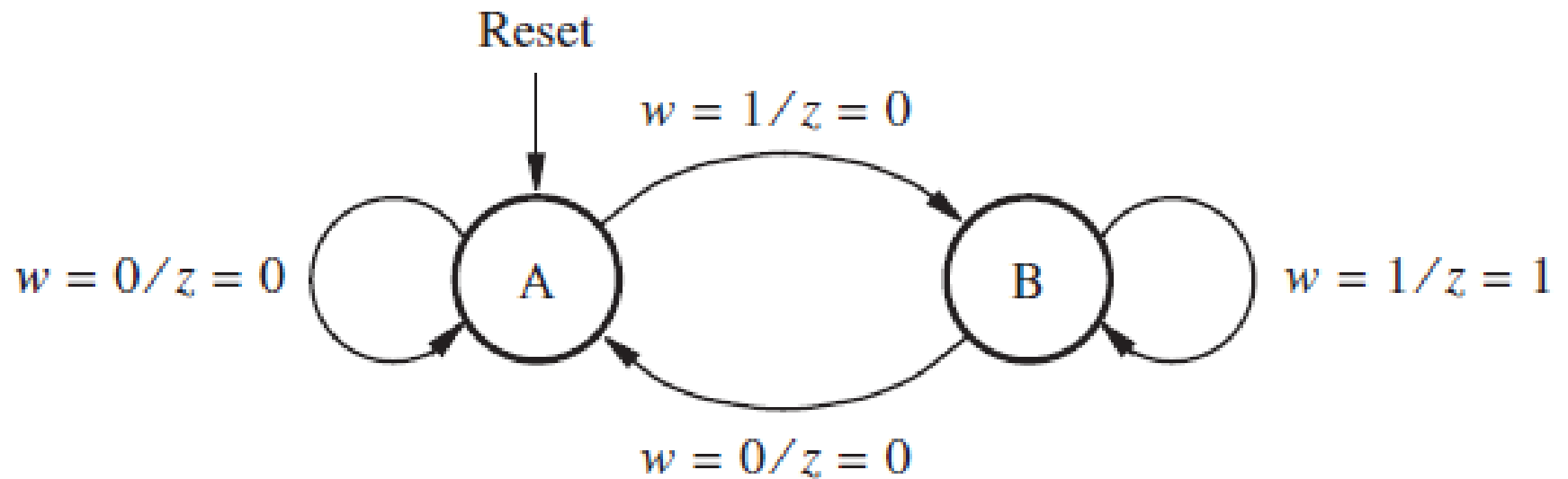
- Previous specs
 - output $z = 1$ in the clock cycle that follows the detection of the second occurrence of $w = 1$

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

- New specs
 - z should be equal to 1 in the same clock cycle when the second occurrence of $w = 1$ is detected

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0

State diagram



State table & Circuit

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Figure 8.24 State table for the FSM in Figure 8.23.

Present state	Next state		Output	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
y	Y	Y	z	z
A	0	1	0	0
B	1	1	0	1

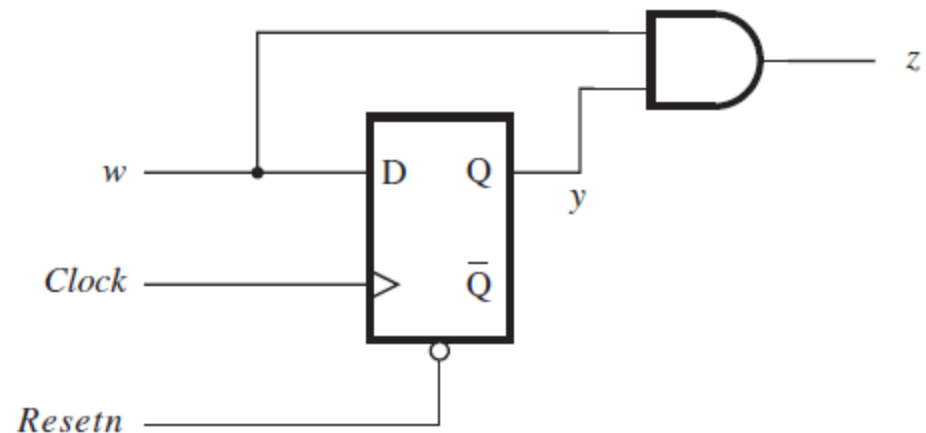
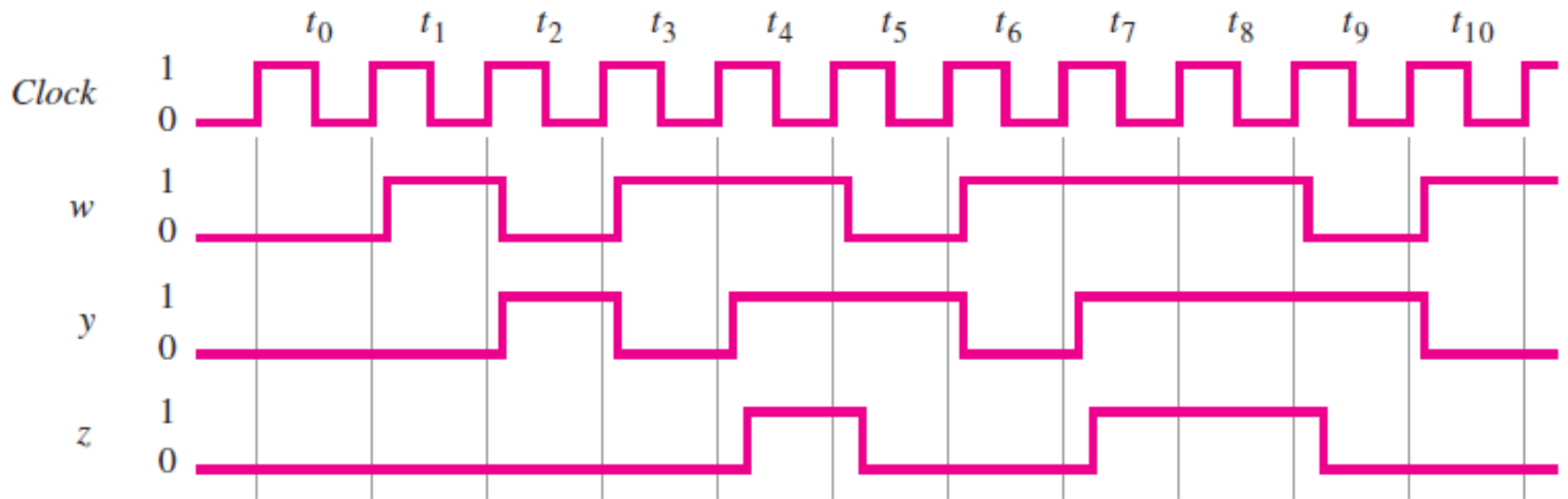


Figure 8.25 State-assigned table for the FSM in Figure 8.24.

Timing diagram



(b) Timing diagram

Verilog code- Mealy

```
module mealy (Clock, Resetn, w, z);  
    input  Clock, Resetn, w;  
    output z;  
    reg y, Y, z;  
    parameter A = 0, B = 1;  
  
    // Define the sequential block  
    always @(negedge Resetn or posedge Clock)  
        if (Resetn == 0) y <= A;  
        else y <= Y;
```

```

// Define the next state and output combinational circuit
always @(w or y)
    case (y)
        A: if (w)
            begin
                z = 0;
                Y = B;
            end
            else
            begin
                z = 0;
                Y = A;
            end
        B: if (w)
            begin
                z = 1;
                Y = B;
            end
            else
            begin
                z = 0;
                Y = A;
            end
    endcase
endmodule

```

Serial Adder Example

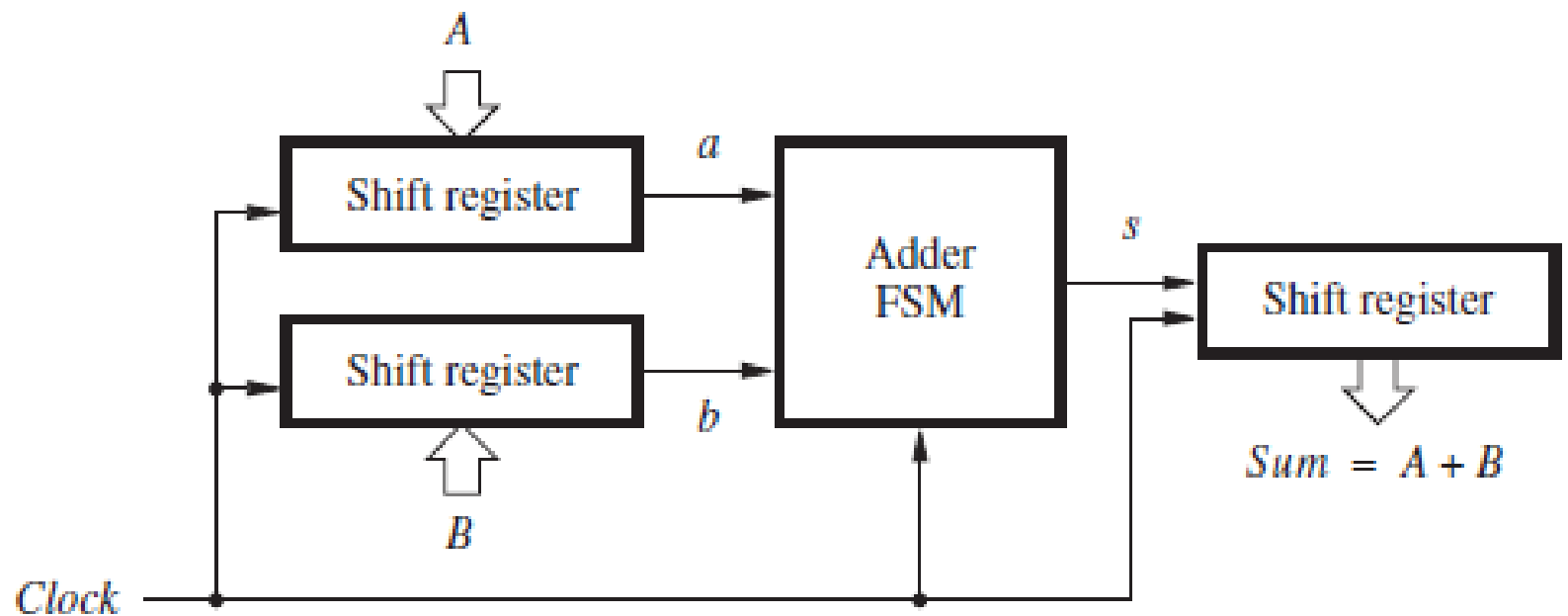


Figure 8.39 Block diagram for the serial adder.

Serial Adder- Mealy FSM

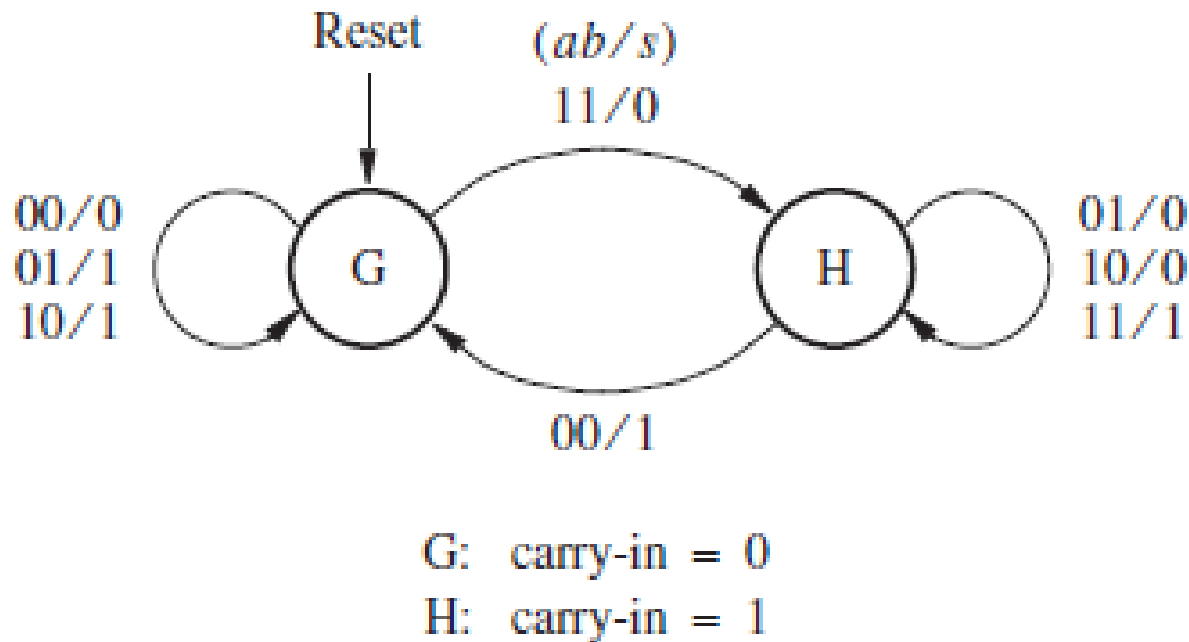


Figure 8.40 State diagram for the serial adder FSM.

$$Y = ab + ay + by \quad \text{Next state eq.}$$

$$s = a \oplus b \oplus y \quad \text{Output eq.}$$

Serial Adder- Mealy FSM

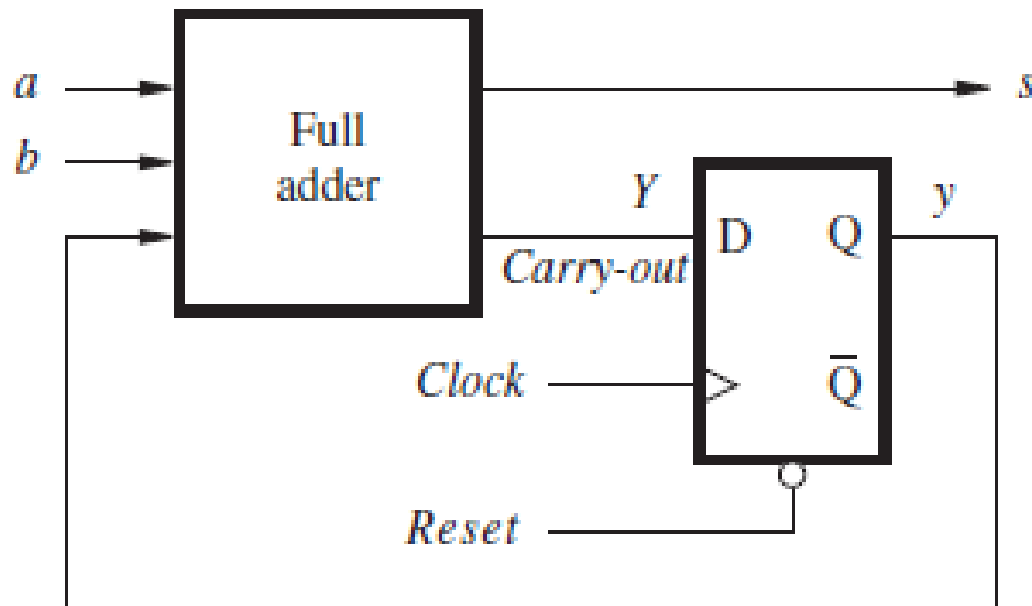
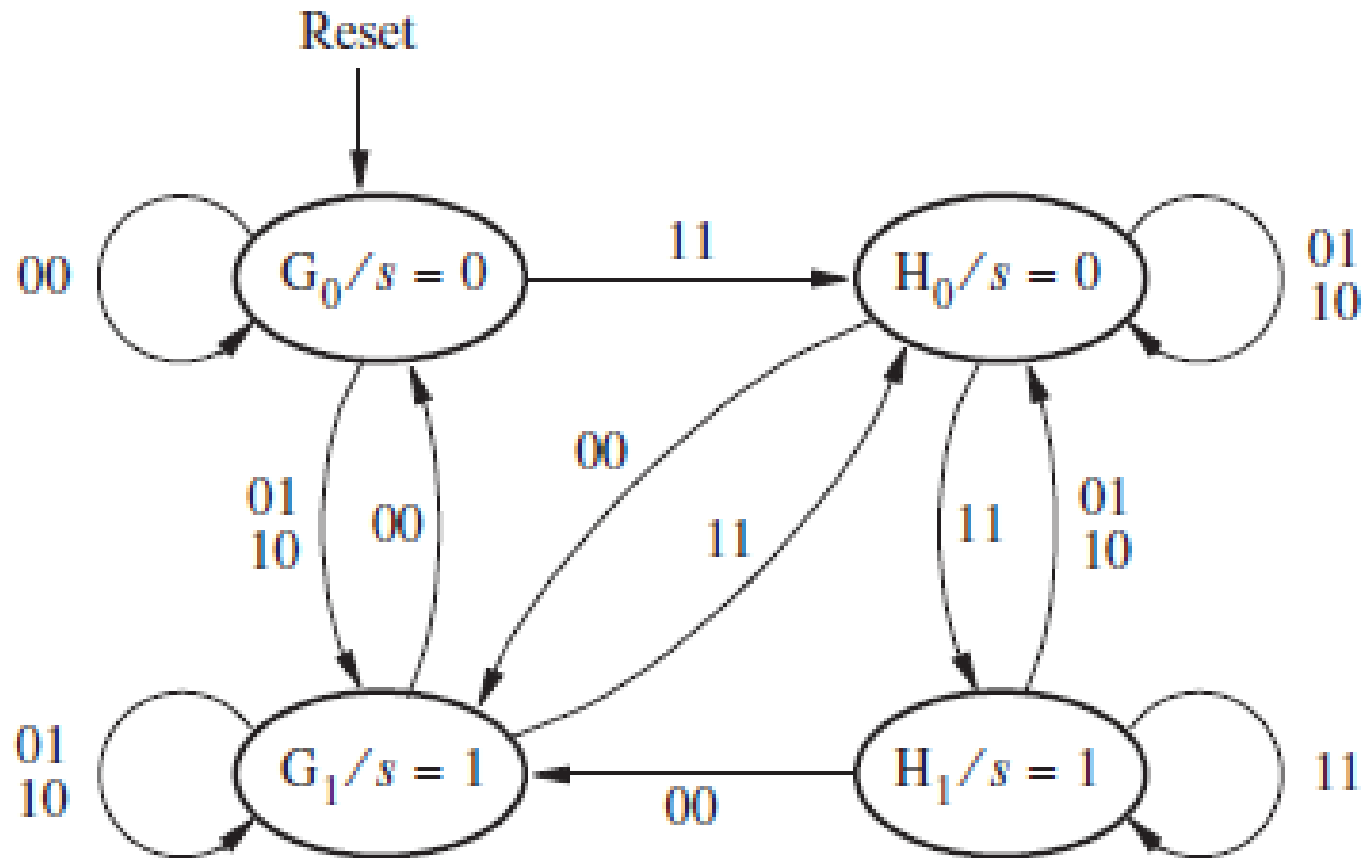


Figure 8.43 Circuit for the adder FSM in Figure 8.39.

Serial Adder- Moore FSM



Serial Adder- Moore FSM

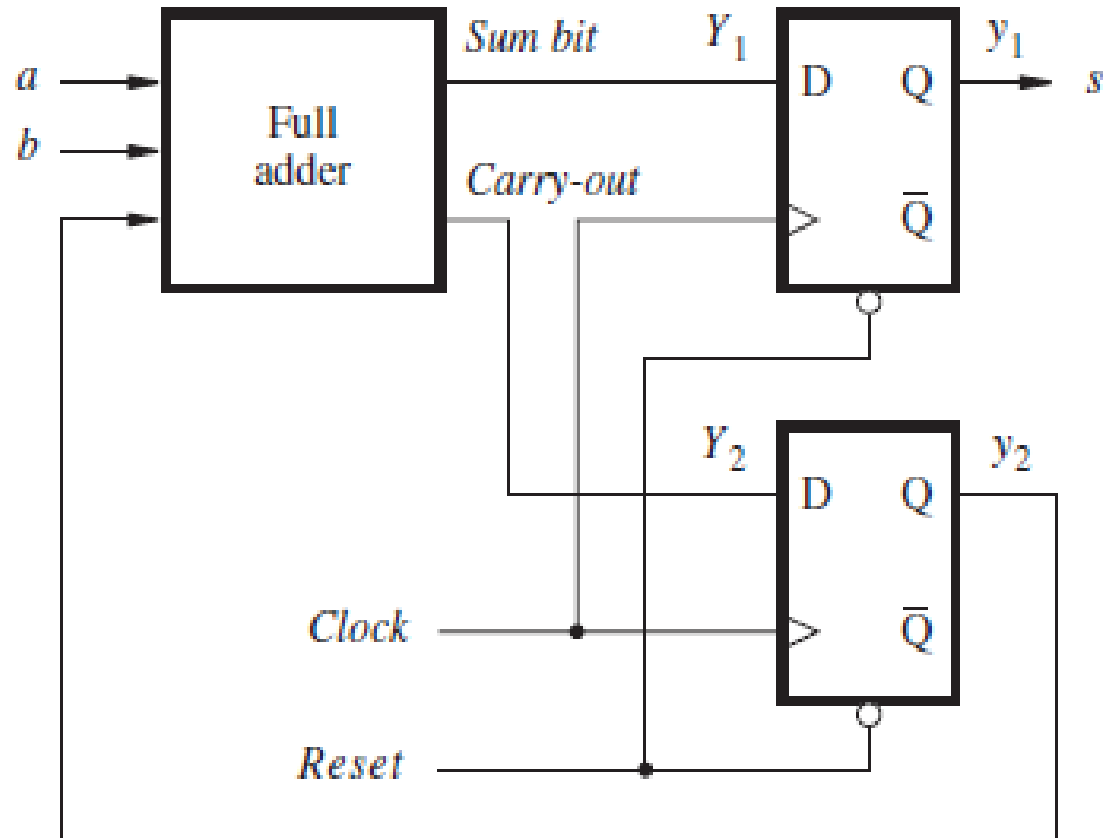


Figure 8.47 Circuit for the Moore-type serial adder FSM.

Counter FSM

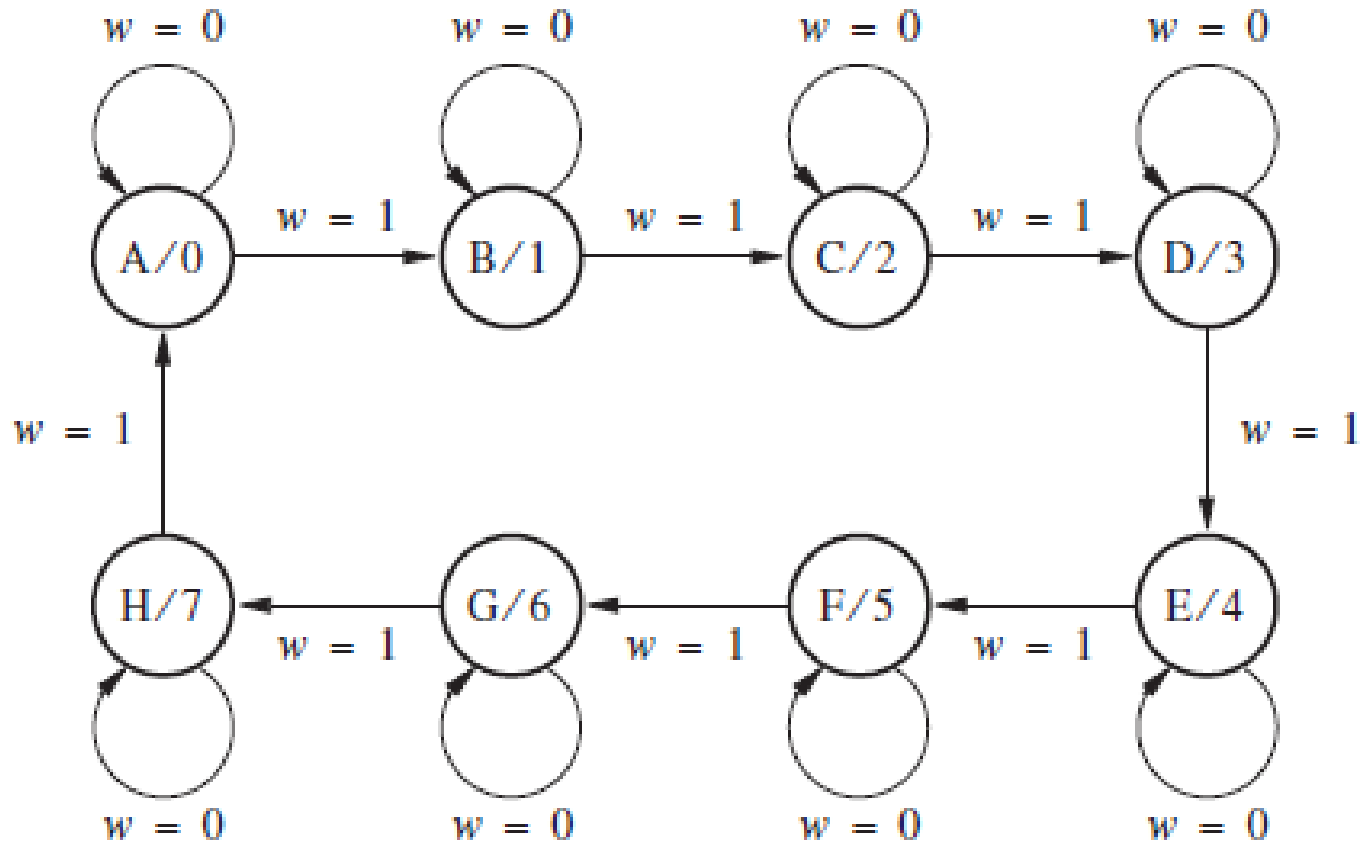


Figure 8.60 State diagram for the counter.

Counter FSM

Present state	Next state		Output
	$w = 0$	$w = 1$	
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7

Figure 8.61 State table for the counter.

One hot encoding

- Uses as many state variables as there are states in a sequential circuit
- for each state all but one of the state variables are equal to 0
- Example

	Present state $y_3y_2y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_3Y_2Y_1$	$Y_3Y_2Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Figure 8.20

One-hot state assignment for the sequential circuit in Figure 8.4.

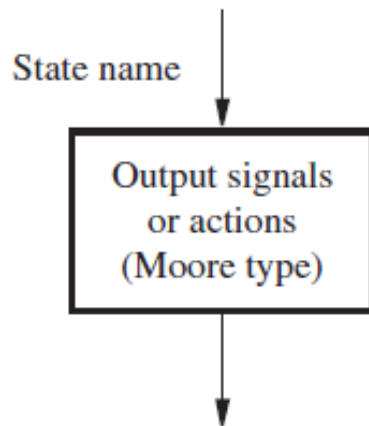
Why state diagrams are not enough

- Not flexible enough for describing very complex finite state machines
- Not suitable for gradual refinement of finite state machine
- Do not obviously describe an algorithm: that is, well specified
- Gradual shift towards program-like representations:
 - Algorithmic State Machine (ASM) Notation
 - Hardware Description Languages (e.g., Verilog, VHDL)

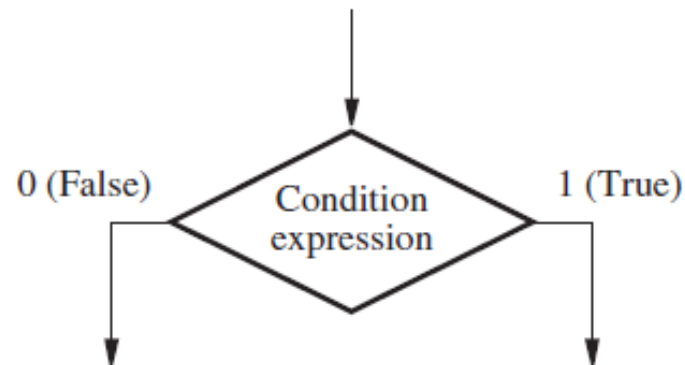
Algorithmic State Machine

- A flowchart-like graphical notation that describes the cycle-by-cycle operations of an algorithm
- Each step takes one clock cycle
- Composed of rectangles, diamonds, ovals, and arrows interconnecting them
- Moore machines do not have ovals
- Mealy machines contain ovals
- Describes behavior rather than structure
- Provides a mechanism for performing systematic step-by-step design
- Can be directly translated to Verilog code
- Used to design synchronous sequential circuits

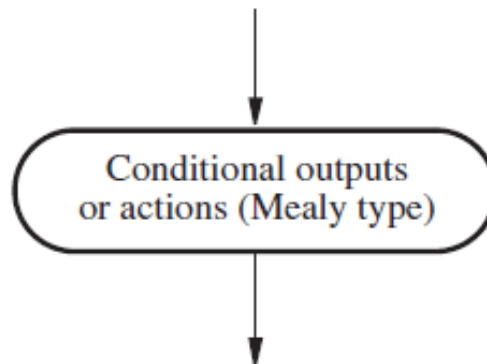
Algorithmic State Machine (ASM) Charts



(a) State box



(b) Decision box



(c) Conditional output box

State box

- It is equivalent to a node in the state diagram or a row in the state table.
- The name of the state is indicated outside the box in the top-left corner
- The Moore-type outputs are listed inside the box

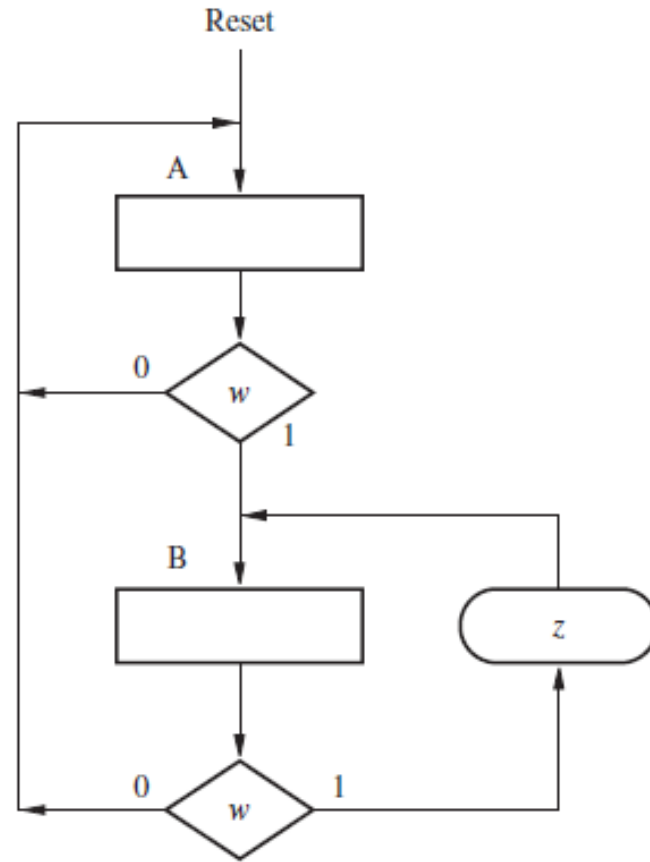
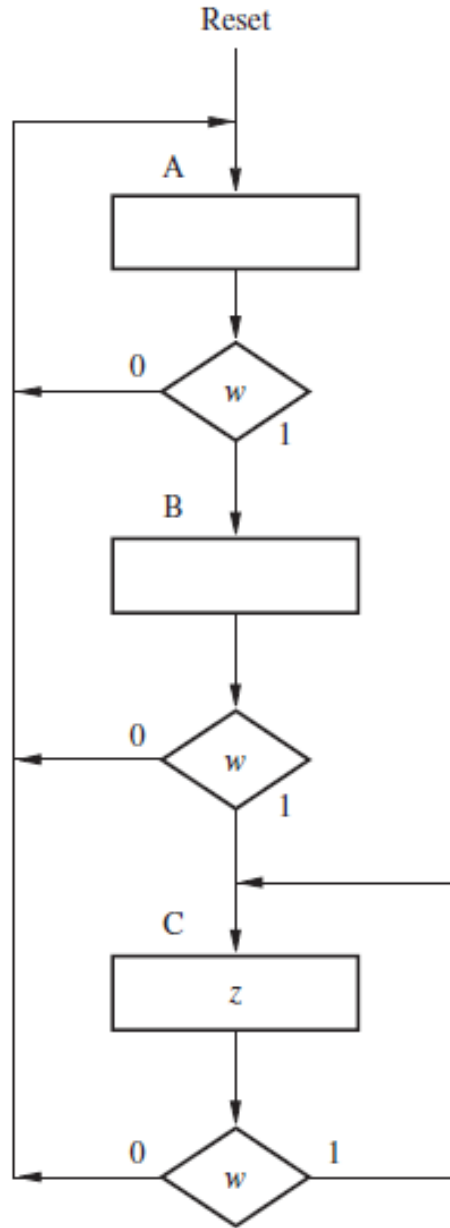
Decision box

- A diamond indicates that the stated condition expression is to be tested and the exit path is to be chosen accordingly
- The condition expression consists of one or more inputs to the FSM

Conditional output box

- An oval denotes the output signals that are of Mealy type
- These outputs depend on the values of the state variables and the inputs of the FSM
- The condition that determines whether such outputs are generated is specified in a decision box

Moore/ Mealy ASM charts



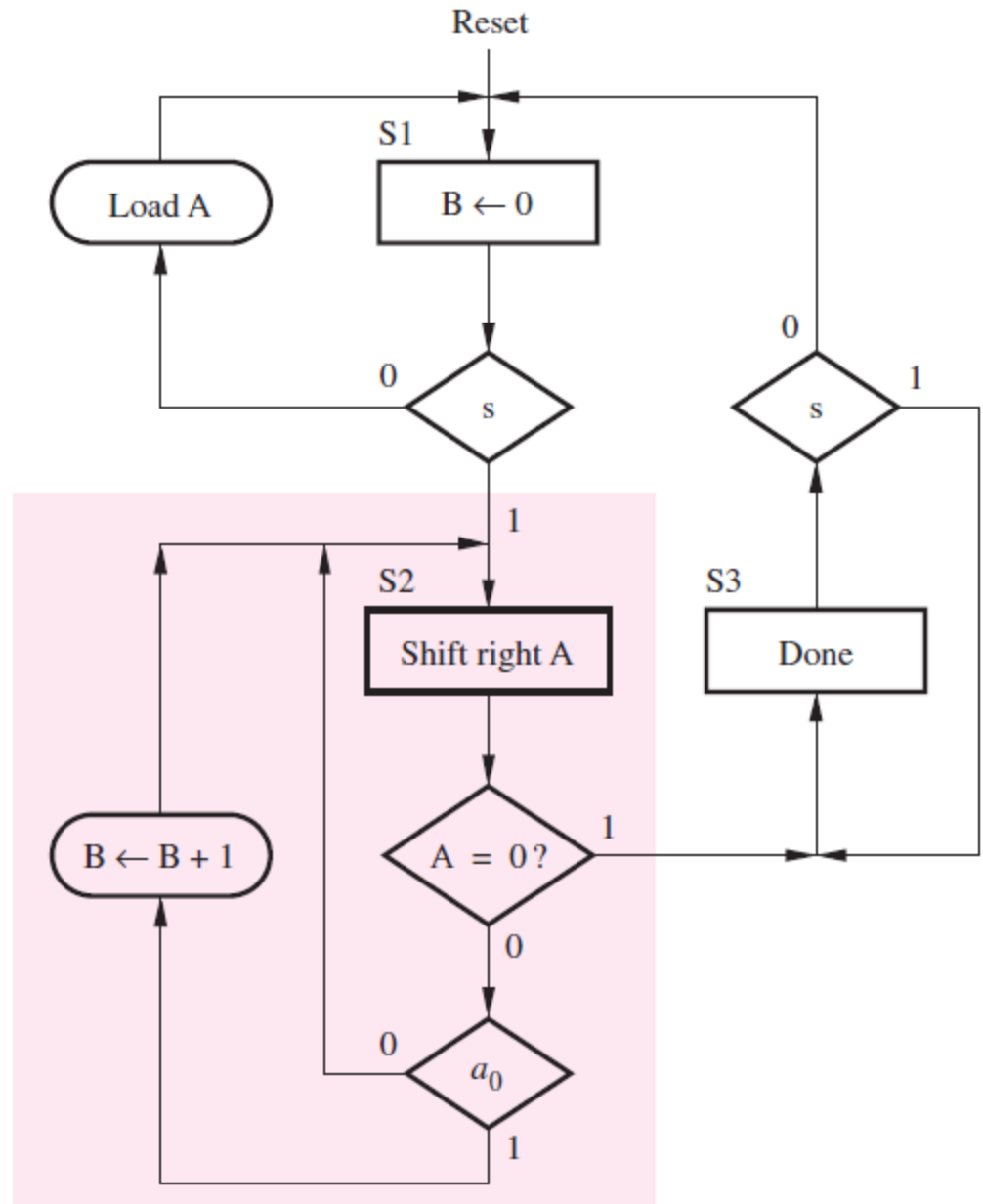
Example: A Bit-Counting Circuit

- pseudo-code

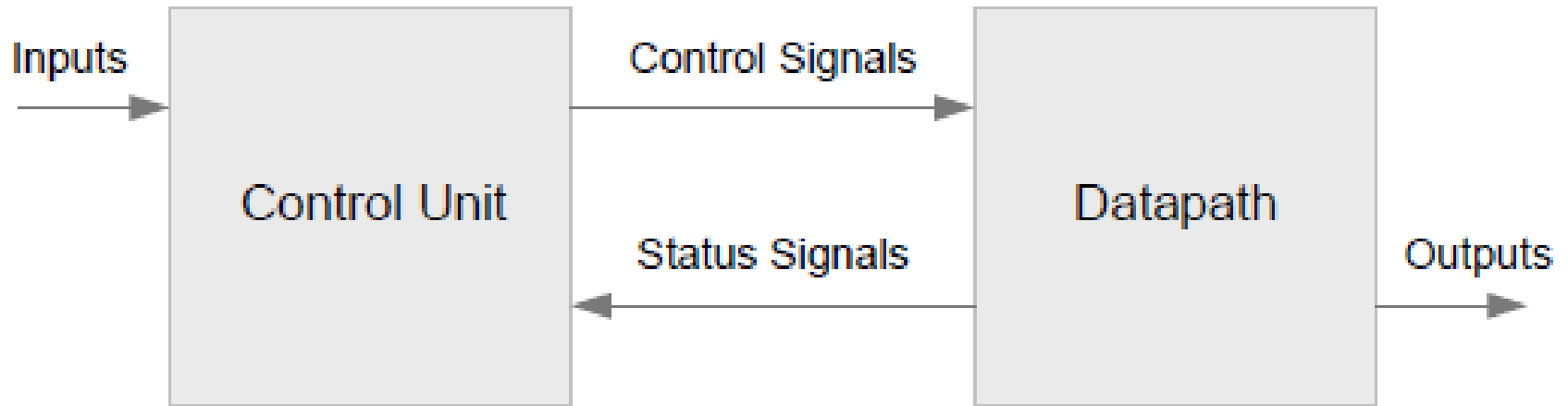
```
 $B = 0 ;$   
 $\text{while } A \neq 0 \text{ do}$   
     $\text{if } a_0 = 1 \text{ then}$   
         $B = B + 1 ;$   
     $\text{end if ;}$   
     $\text{Right-shift } A ;$   
 $\text{end while ;}$ 
```

Pseudo code based ASM chart

$B = 0$;
while $A \neq 0$ do
 if $a_0 = 1$ then
 $B = B + 1$;
 end if ;
 Right-shift A ;
end while ;



Data-path and control circuits



Datapath- Bit counting example

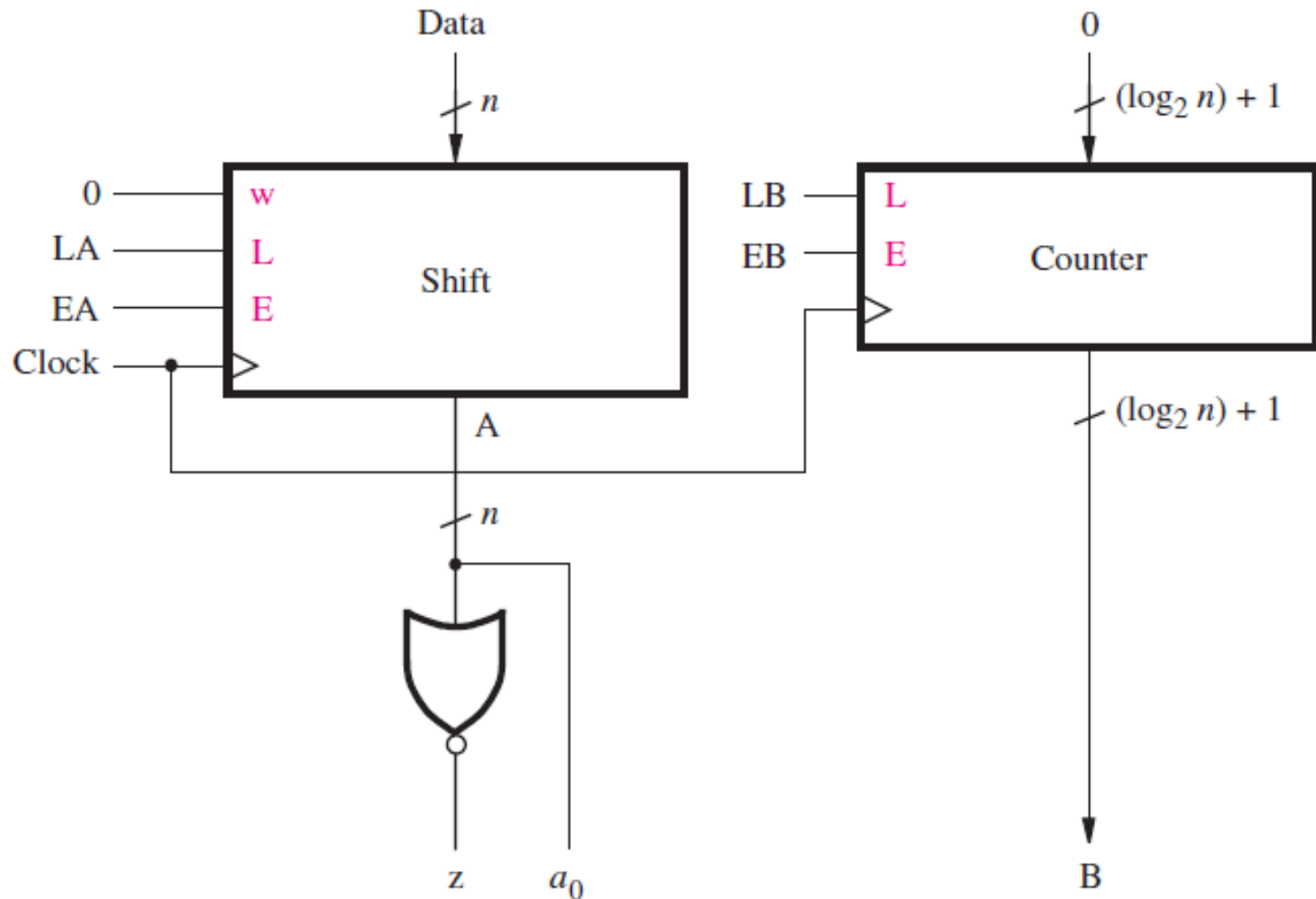


Figure 10.11 Datapath for the ASM chart in Figure 10.10.

ASM chart- Bit counting example

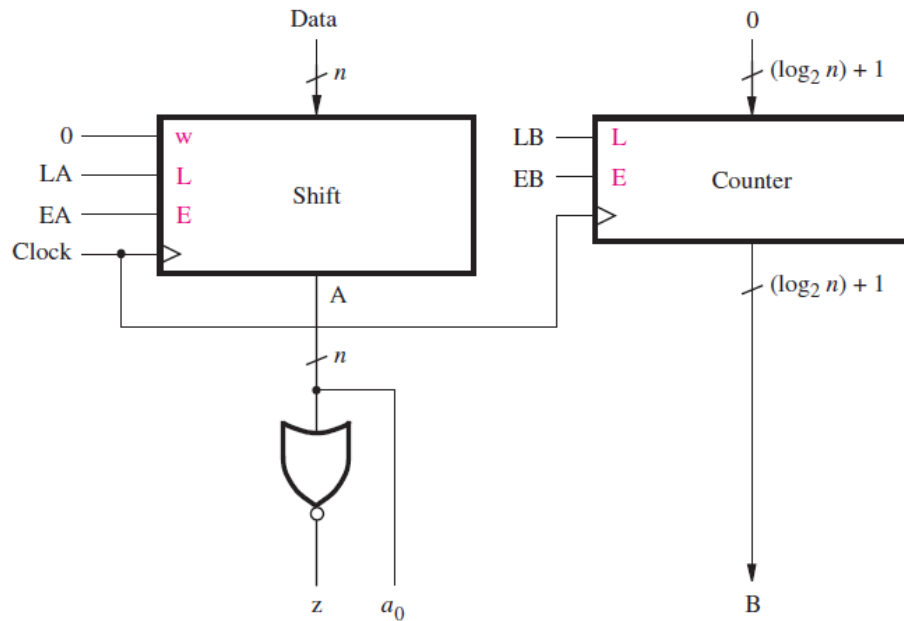


Figure 10.11 Datapath for the ASM chart in Figure 10.10.

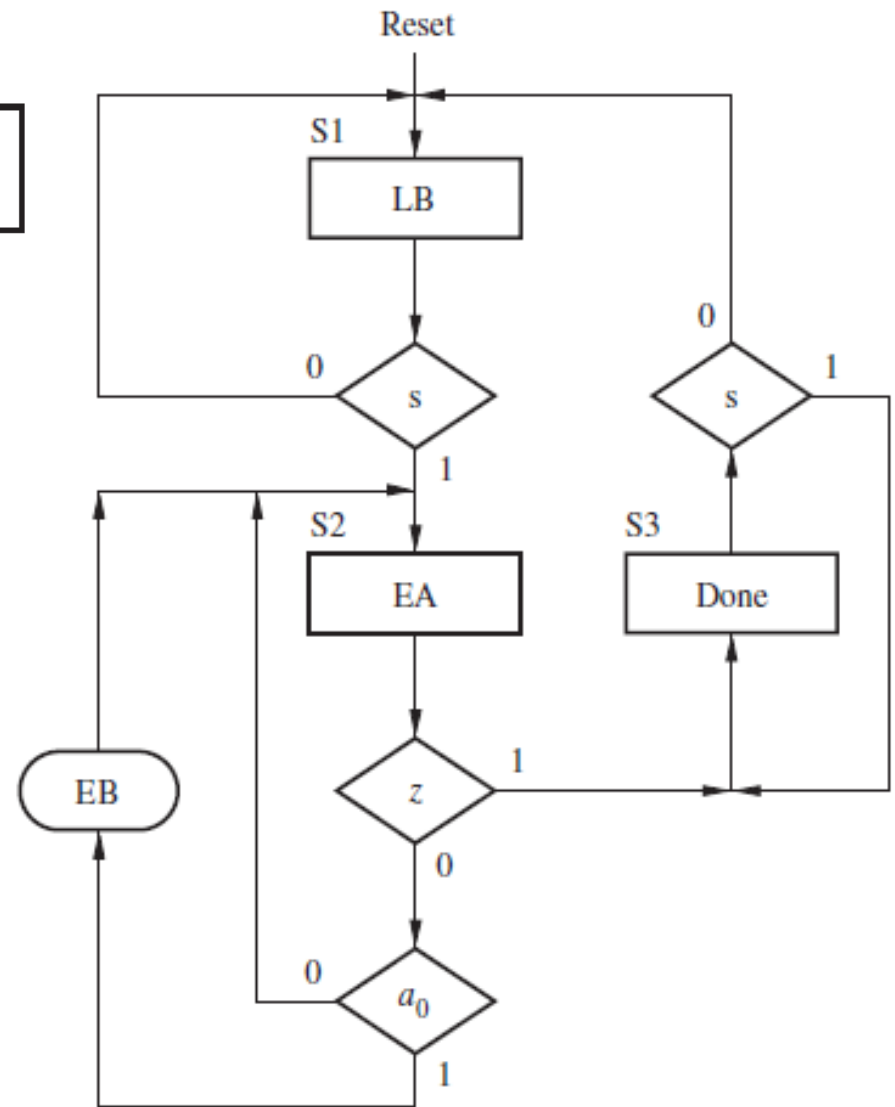
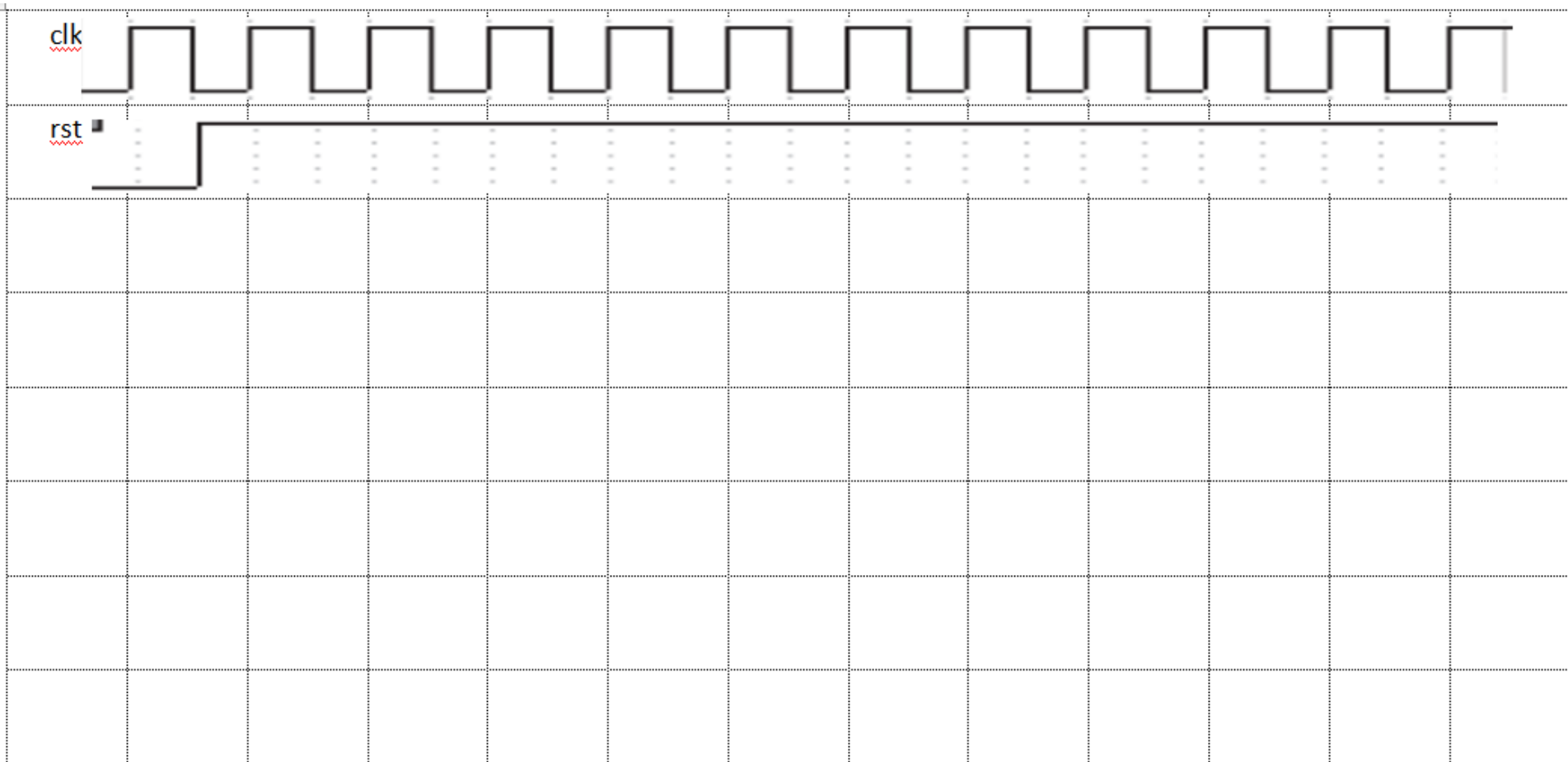


Figure 10.12 ASM chart for the bit counter datapath circuit.

Timing diagram



Simulation results

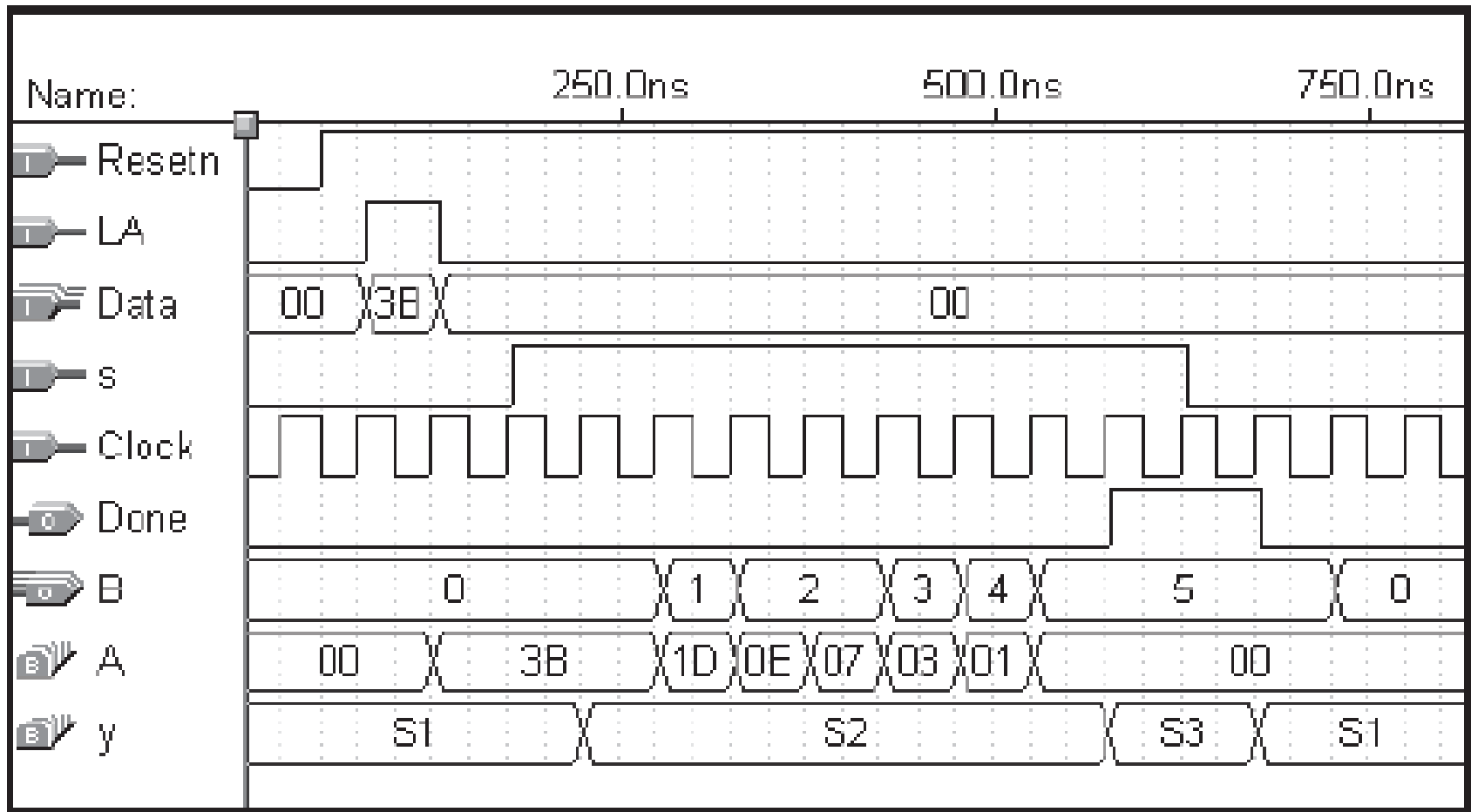


Figure 10.14 Simulation results for the bit-counting circuit.

Verilog code- Bit counting FSM

```
module bitcount (Clock, Resetn, LA, s, Data, B, Done);  
    input  Clock, Resetn, LA, s;  
    input  [7:0] Data;  
    output [3:0] B;  
    output Done;  
    wire [7:0] A;  
    wire z;  
    reg [1:0] Y, y;  
    reg [3:0] B;  
    reg Done, EA, EB, LB;
```

- Verilog
code, Bit
counting
FSM

```
// control circuit
```

```
parameter S1 = 2'b00, S2 = 2'b01, S3 = 2'b10;
```

```
always @(s or y or z)
```

```
begin: State_table
```

```
  case (y)
```

```
    S1: if (!s) Y = S1;
```

```
        else Y = S2;
```

```
    S2: if (z == 0) Y = S2;
```

```
        else Y = S3;
```

```
    S3: if (s) Y = S3;
```

```
        else Y = S1;
```

```
    default: Y = 2'bxx;
```

```
  endcase
```

```
end
```

```
always @(posedge Clock or negedge Resetn)
```

```
begin: State_flipflops
```

```
  if (Resetn == 0)
```

```
    y <= S1;
```

```
  else
```

```
    y <= Y;
```

```
end
```

- Verilog
code, Bit
counting
FSM

```
always @(y or A[0])
begin: FSM_outputs
    // defaults
    EA = 0; LB = 0; EB = 0; Done = 0;
    case (y)
        S1: LB = 1;
        S2: begin
            EA = 1;
            if (A[0]) EB = 1;
            else EB = 0;
        end
        S3: Done = 1;
    endcase
end
```

- Verilog
code, Bit
counting
FSM

```
// datapath circuit
```

```
// counter B
```

```
always @(negedge Resetn or posedge Clock)
```

```
    if (!Resetn)
```

```
        B <= 0;
```

```
    else if (LB)
```

```
        B <= 0;
```

```
    else if (EB)
```

```
        B <= B + 1;
```

```
shiftrne ShiftA (Data, LA, EA, 0, Clock, A);
```

```
assign  z = ~| A;
```

```
endmodule
```

- Right to left
shift
register

```
module shiftlne (R, L, E, w, Clock, Q);  
    parameter n = 4;  
    input [n-1:0] R;  
    input L, E, w, Clock;  
    output [n-1:0] Q;  
    reg [n-1:0] Q;  
    integer k;  
  
    always @(posedge Clock)  
    begin  
        if (L)  
            Q <= R;  
        else if (E)  
            begin  
                Q[0] <= w;  
                for (k = 1; k < n; k = k+1)  
                    Q[k] <= Q[k-1];  
            end  
        end  
    end  
  
endmodule
```

Shift-and-Add Multiplier

Decimal	Binary	
13	1 1 0 1	Multiplicand
$\times 11$	$\times 1 0 1 1$	Multiplier
13	1 1 0 1	
13↓	1 1 0 1	
	0 0 0 0	
143	1 1 0 1	
	↓ ↓ ↓	
	1 0 0 0 1 1 1 1	Product

(a) Manual method

```

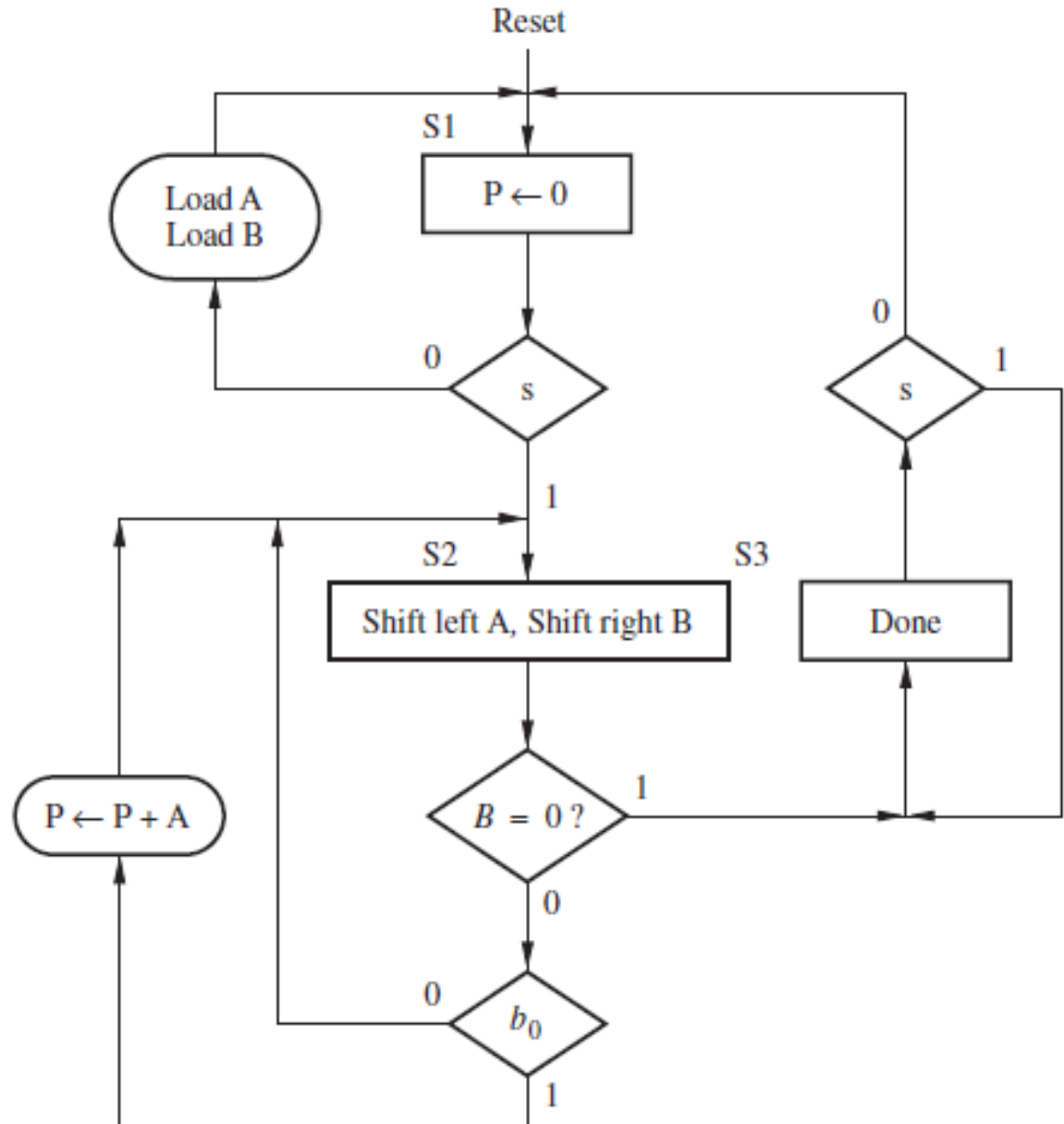
P = 0 ;
for i = 0 to n - 1 do
    if  $b_i = 1$  then
         $P = P + A$  ;
    end if ;
    Left-shift A ;
end for ;
    
```

(b) Pseudo-code

ASM chart for multiplier

$P = 0$;
for $i = 0$ to $n - 1$ do
 if $b_i = 1$ then
 $P = P + A$;
 end if ;
 Left-shift A ;
end for ;

(b) Pseudo-code

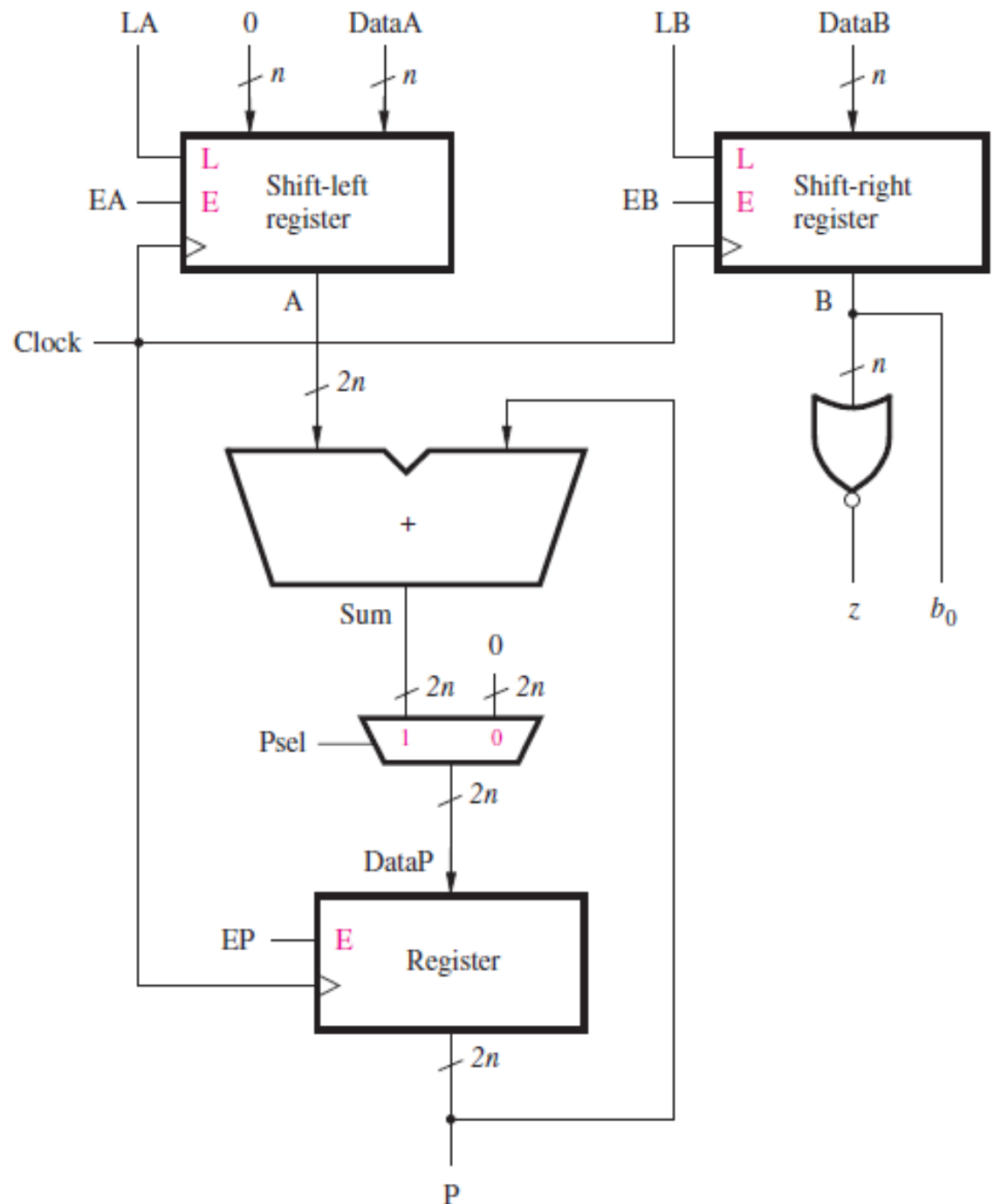


- Datapath

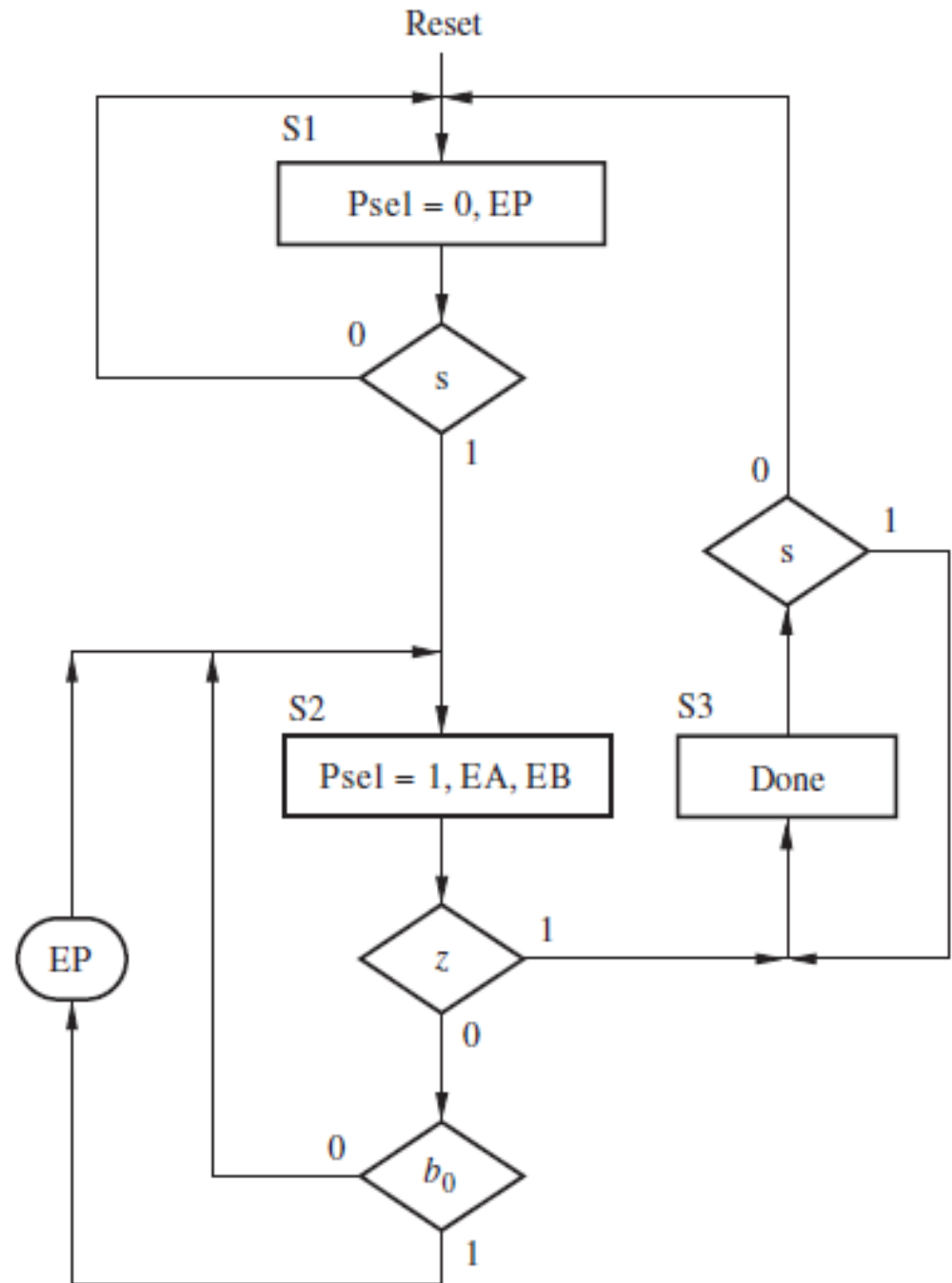
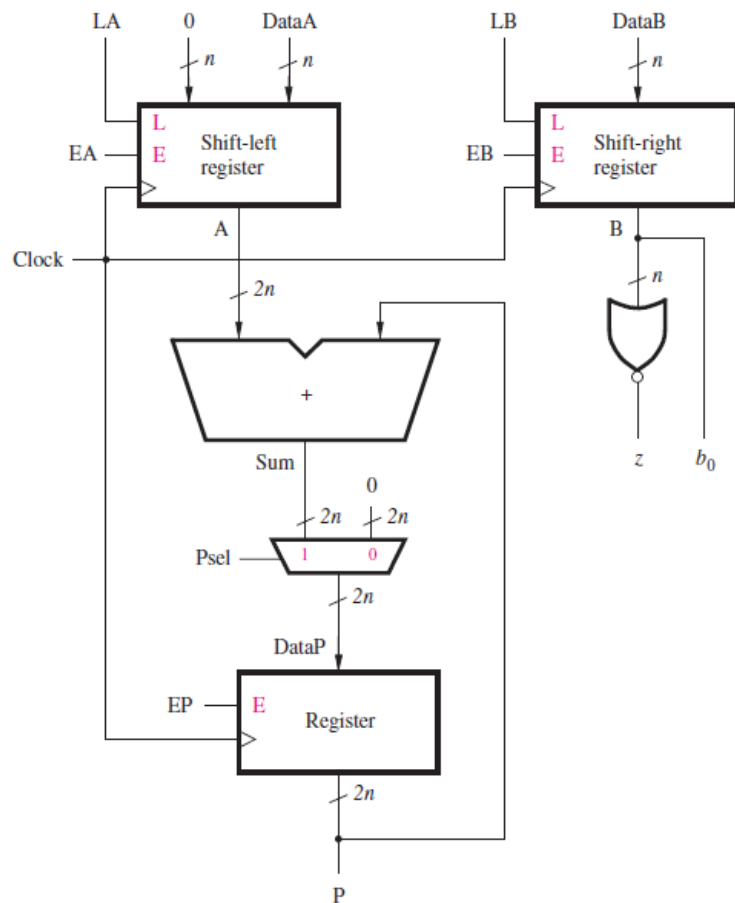
```

P = 0 ;
for i = 0 to n - 1 do
    if bi = 1 then
        P = P + A ;
    end if ;
    Left-shift A ;
end for ;
  
```

(b) Pseudo-code



- ASM chart



Multiplier code

```
module multiply (Clock, Resetn, LA, LB, s, DataA, DataB, P, Done);  
    parameter n = 8;  
    input  Clock, Resetn, LA, LB, s;  
    input  [n-1:0] DataA, DataB;  
    output [n+n-1:0] P;  
    output Done;  
    wire z;  
    reg [n+n-1:0] A, DataP;  
    wire [n+n-1:0] Sum;  
    reg [1:0] y, Y;  
    reg [n-1:0] B;  
    reg Done, EA, EB, EP, Psel;  
    integer k;
```

Multiplier code

- Control unit

```
parameter S1 = 2'b00, S2 = 2'b01, S3 = 2'b10;
```

```
always @(s or y or z)
```

```
begin: State_table
```

```
    case (y)
```

```
        S1: if (s == 0) Y = S1;  
            else Y = S2;
```

```
        S2: if (z == 0) Y = S2;  
            else Y = S3;
```

```
        S3: if (s == 1) Y = S3;  
            else Y = S1;
```

```
        default: Y = 2'bxx;
```

```
    endcase
```

```
end
```

```
always @(posedge Clock or negedge Resetn)
```

```
begin: State_flipflops
```

```
    if (Resetn == 0)
```

```
        y <= S1;
```

```
    else
```

```
        y <= Y;
```

```
    end
```

FSM outputs

```
always @(s or y or B[0])
begin: FSM_outputs
    // defaults
    EA = 0; EB = 0; EP = 0; Done = 0; Psel = 0;
    case (y)
        S1: EP = 1;
        S2: begin
            EA = 1; EB = 1; Psel = 1;
            if (B[0]) EP = 1;
            else EP = 0;
        end
        S3: Done = 1;
    endcase
end
```

Data-path circuit

```
//datapath circuit
```

```
shiftrne ShiftB (DataB, LB, EB, 0, Clock, B);
```

```
    defparam ShiftB.n = 8;
```

```
shiftrne ShiftA ({ {n{1'b0}} , DataA }, LA, EA, 0, Clock, A);
```

```
    defparam ShiftA.n = 16;
```

```
assign z = (B == 0);
```

```
assign Sum = A + P;
```

```
// define the 2n 2-to-1 multiplexers
```

```
always @(Psel or Sum)
```

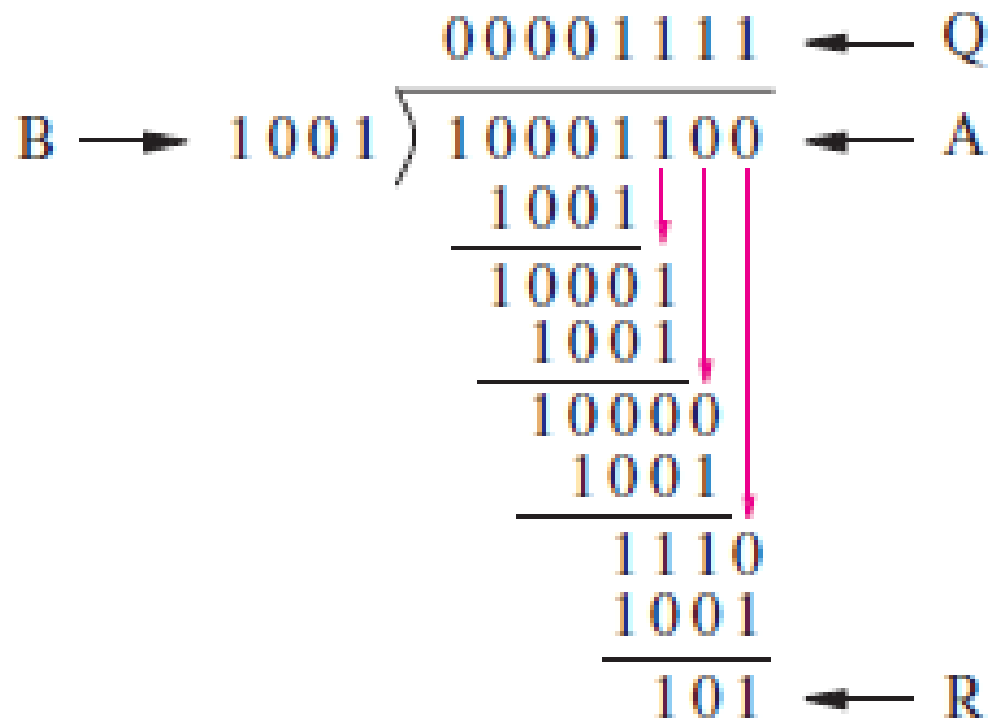
```
    for (k = 0; k < n+n; k = k+1)
```

```
        DataP[k] = Psel ? Sum[k] : 0;
```

```
regne RegP (DataP, Clock, Resetn, EP, P);
```

```
    defparam RegP.n = 16;
```

Divider



```

R = 0 ;
for i = 0 to n - 1 do
    Left-shift R||A ;
    if R ≥ B then
        qi = 1 ;
        R = R - B ;
    else
        qi = 0 ;
    end if ;
end for ;
    
```

(c) Pseudo-code

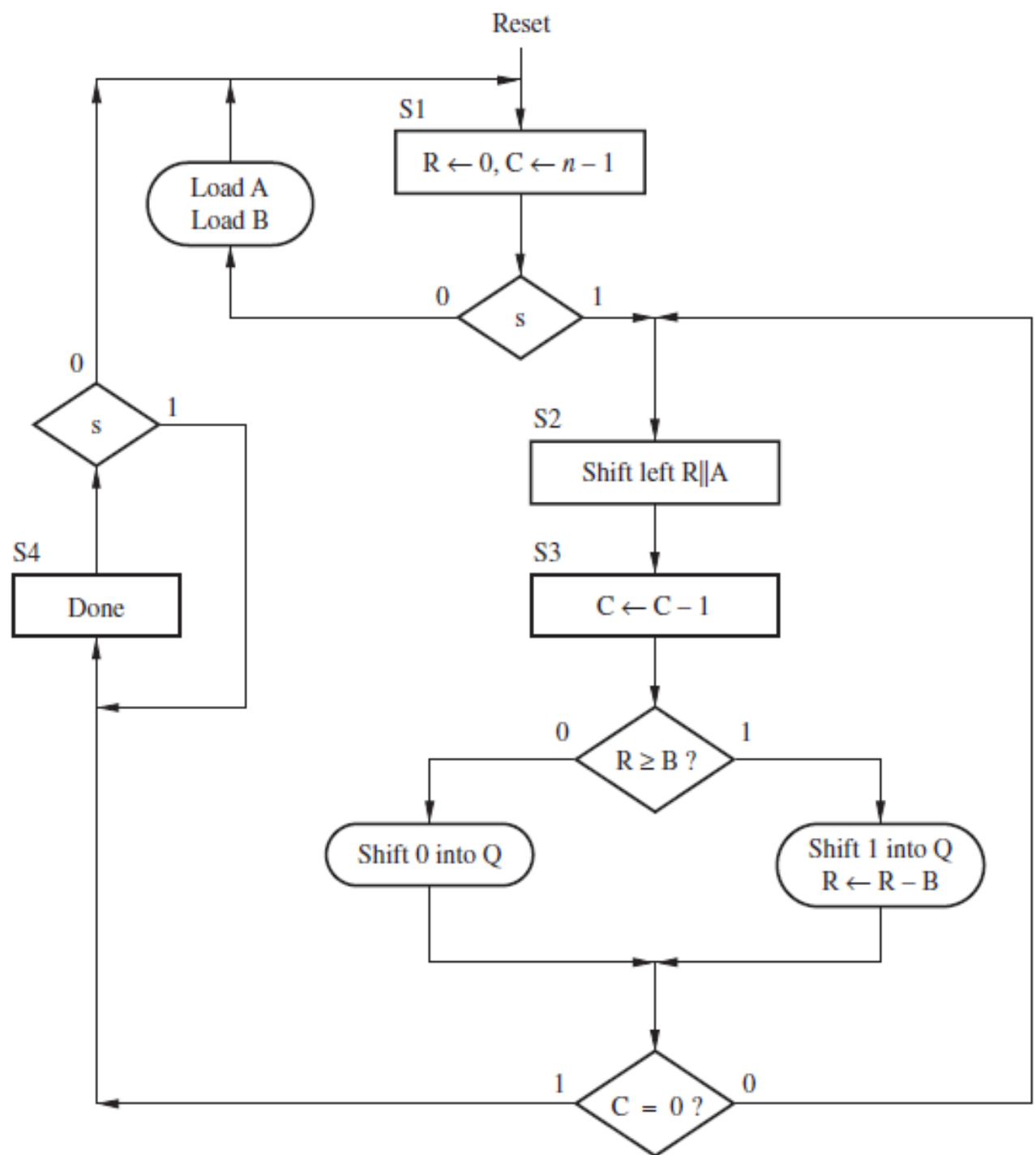
- ASM
- Divider

```

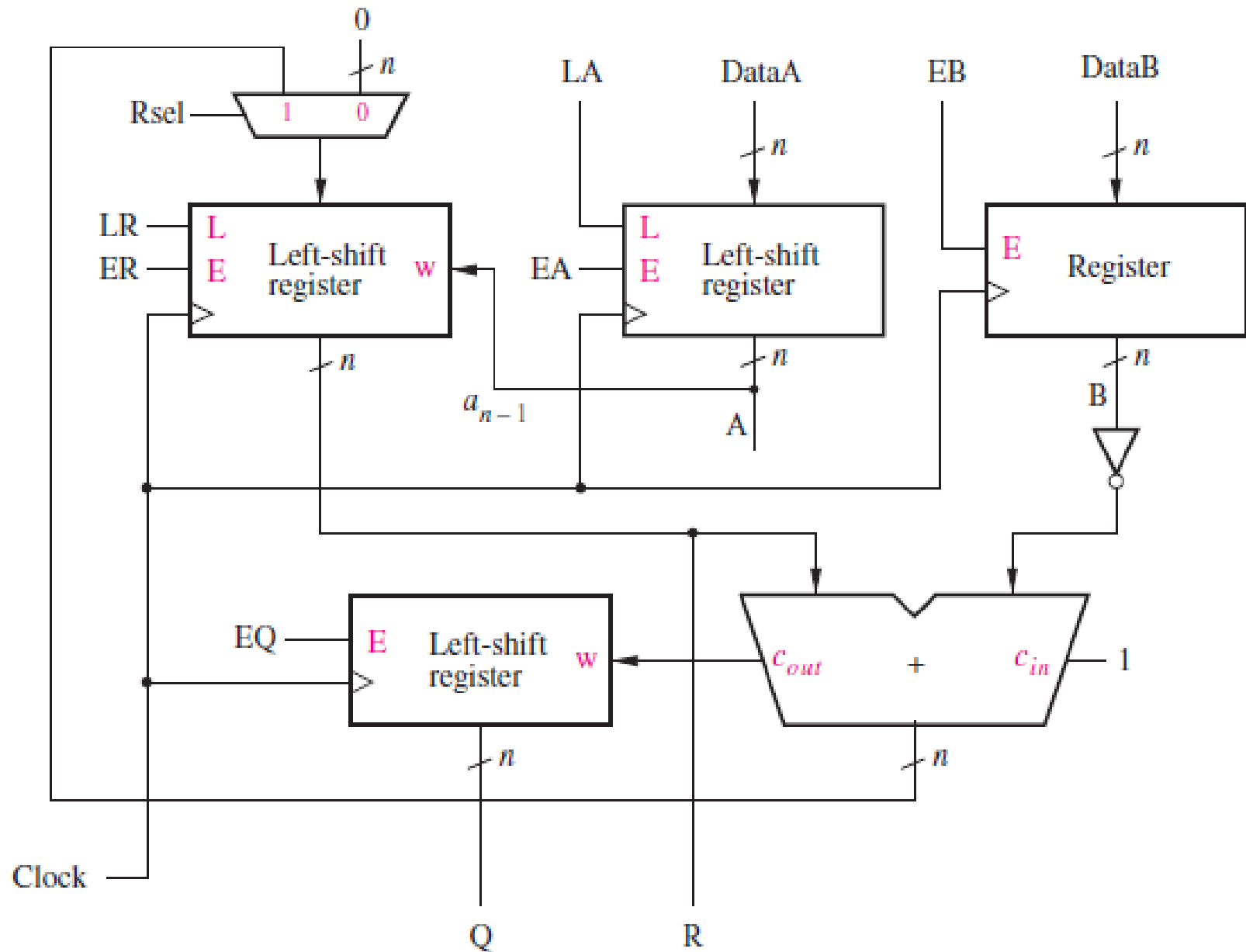
 $R \leftarrow 0$  ;
for  $i = 0$  to  $n - 1$  do
    Left-shift  $R \parallel A$  ;
    if  $R \geq B$  then
         $q_i = 1$  ;
         $R \leftarrow R - B$  ;
    else
         $q_i = 0$  ;
    end if ;
end for ;

```

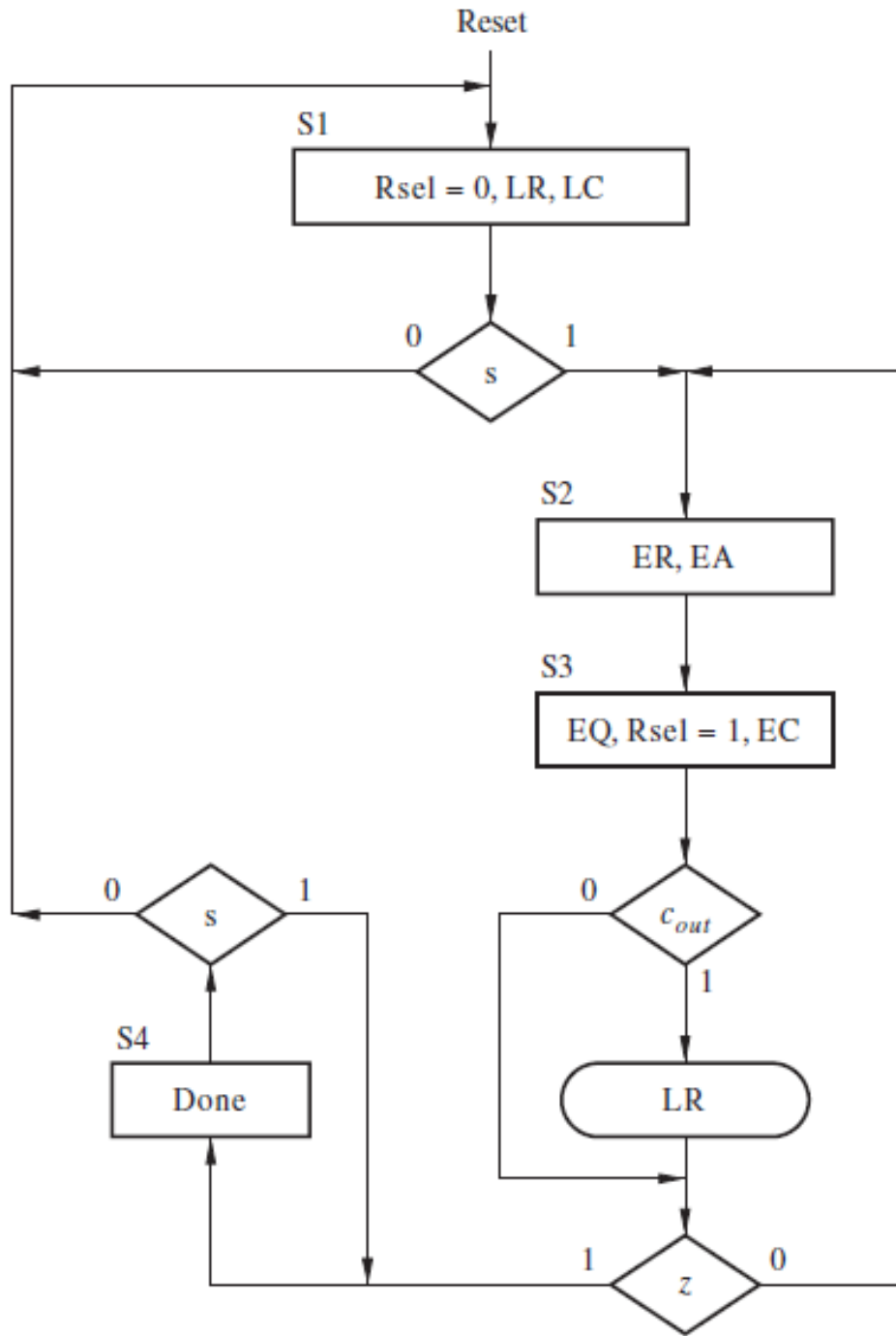
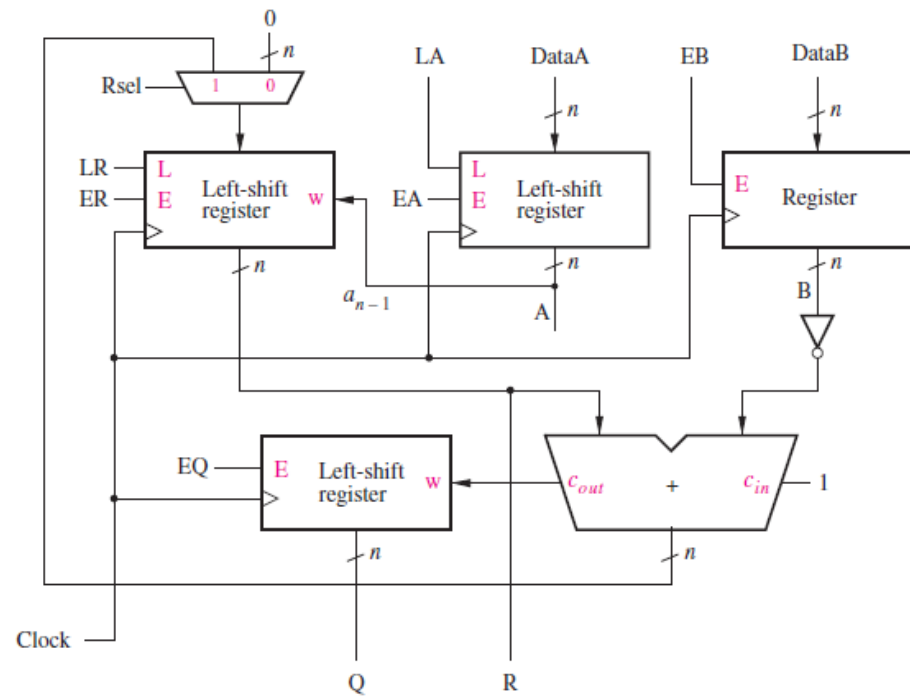
(c) Pseudo-code



Data path: divider



- ASM chart, Control signals of data-path



Square-root Approximation

- State machine based design for Euclidean distance calculation
 - Multi-threaded architecture
 - Resource optimization

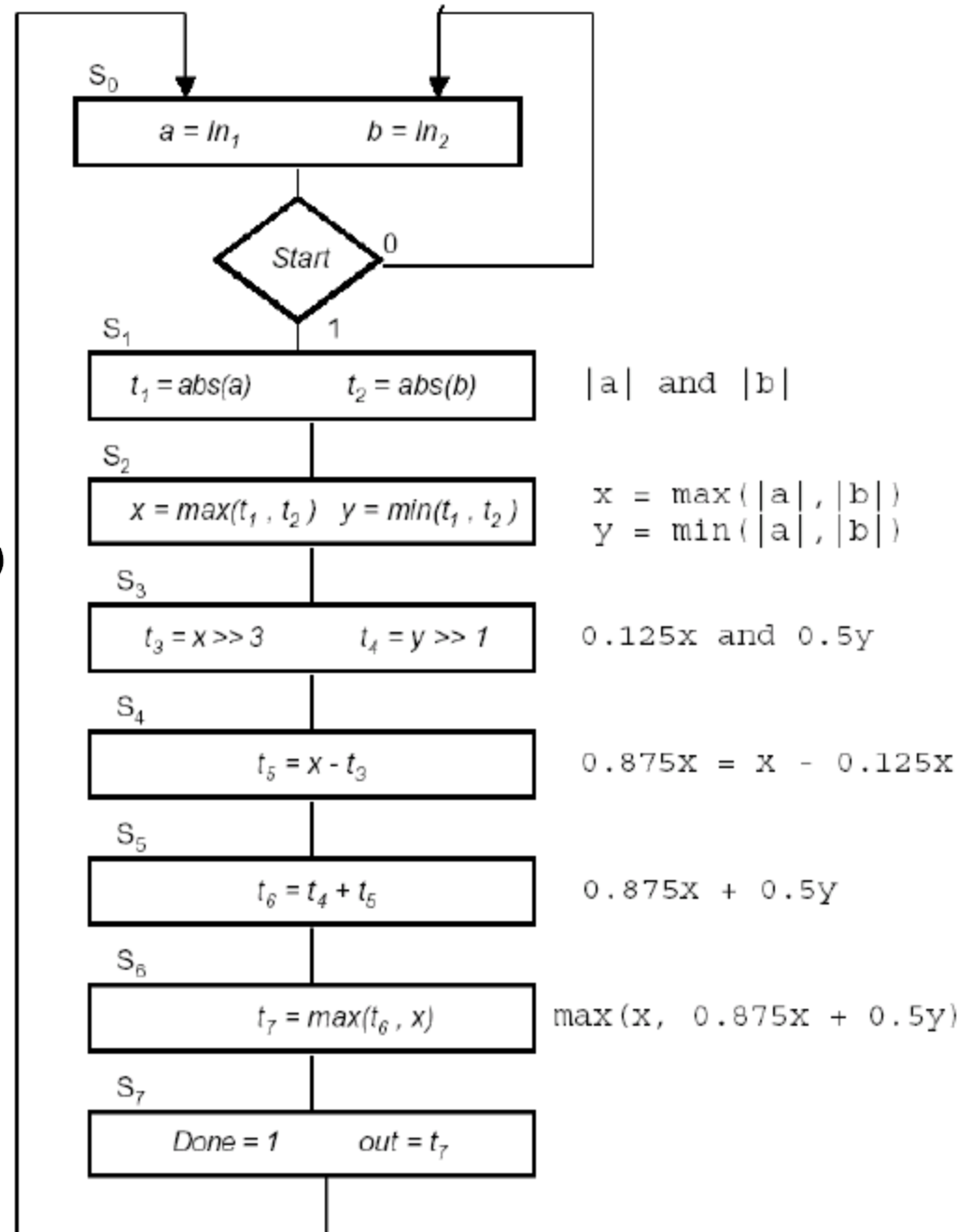
$$\text{square root } (a^2 + b^2) \approx \max((0.875x + 0.5y), x)$$

where

$$x = \max(\text{abs}(a), \text{abs}(b)) \text{ and } y = \min(\text{abs}(a), \text{abs}(b))$$

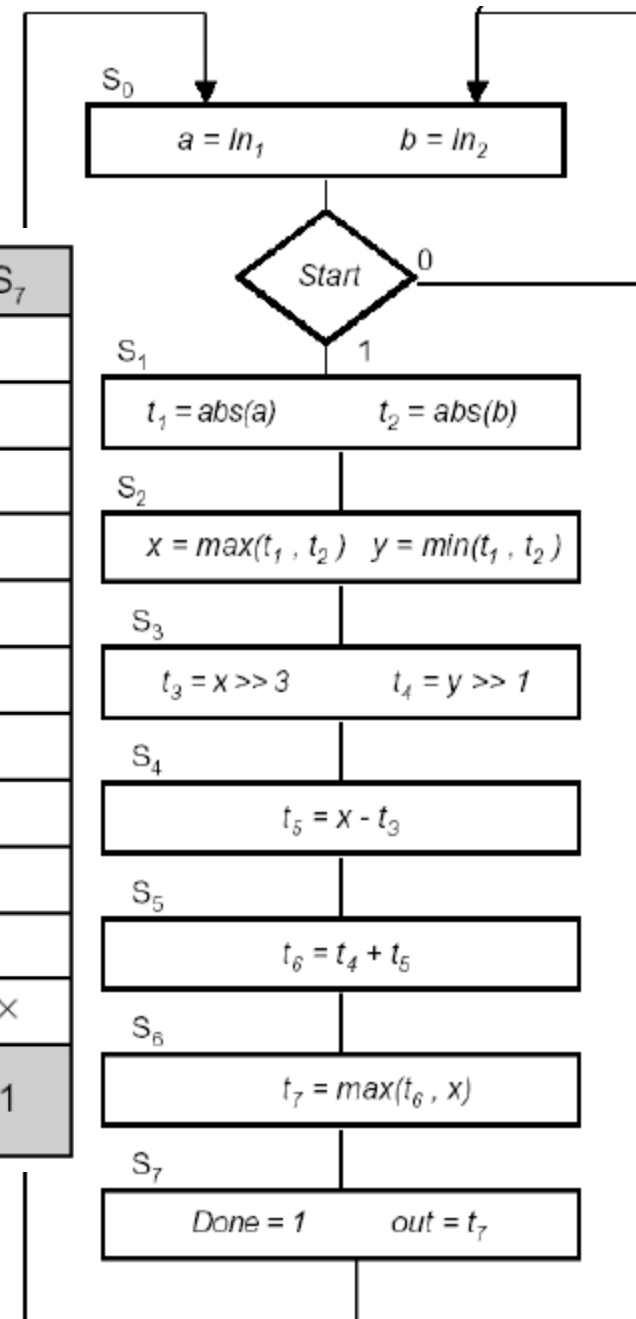
ASM

$\text{square root } (a^2 + b^2) \approx \max((0.875x + 0.5y), x)$
 $x = \max(\text{abs}(a), \text{abs}(b))$
 $y = \min(\text{abs}(a), \text{abs}(b))$



Variable Usage chart

	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
a		×						
b		×						
t_1			×					
t_2			×					
x				×	×	×	×	
y				×				
t_3					×			
t_4					×	×		
t_5						×		
t_6							×	
t_7								×
Number of live variables	0	2	2	2	3	3	2	1



Register assignment

	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
a		x						
b		x						
t_1			x					
t_2			x					
x				x	x	x	x	
y				x				
t_3					x			
t_4					x	x		
t_5						x		
t_6							x	
t_7								x
Number of live variables	0	2	2	2	3	3	2	1

$$R_1 = [a, t_1, x, t_7]$$

$$R_2 = [b, t_2, y, t_3, t_5, t_6]$$

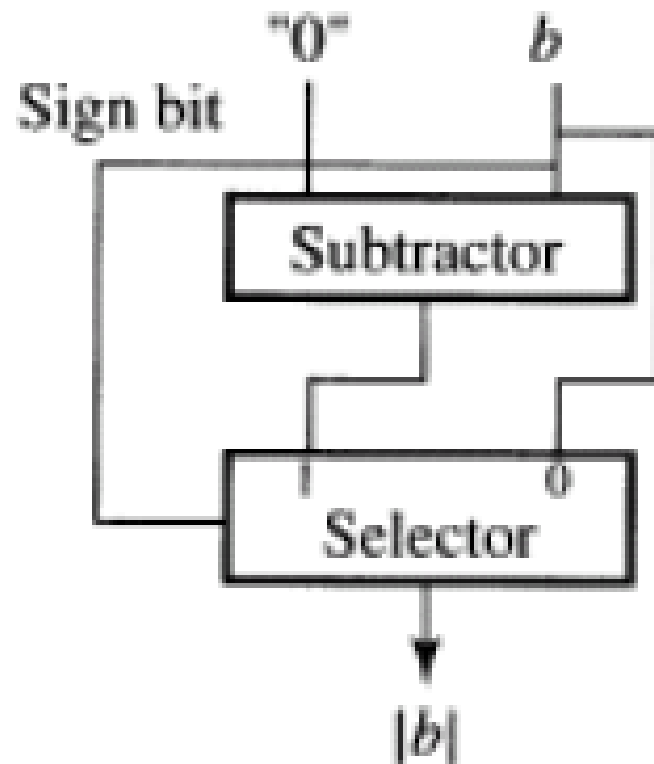
$$R_3 = [t_4]$$

Operation usage

	S ₀	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇	Max number of units
<i>abs</i>		2							2
<i>min</i>			1						1
<i>max</i>			1				1		1
>>				2					2
-					1				1
+						1			1
Number of operations	0	2	2	2	1	1	1		

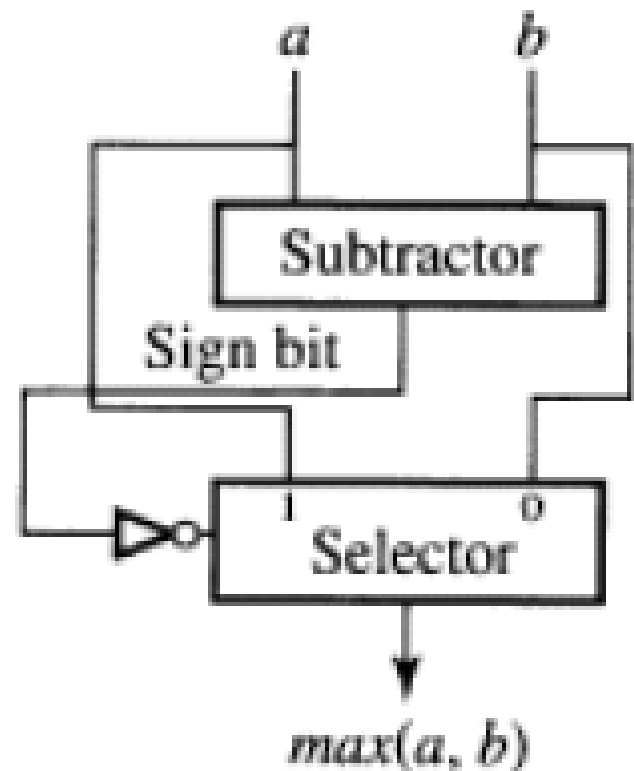
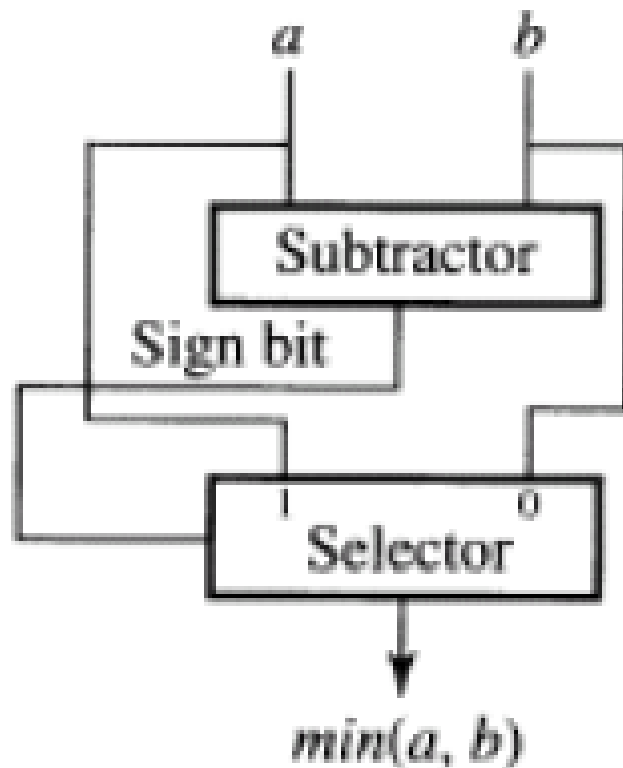
Computation units

- Absolute value unit



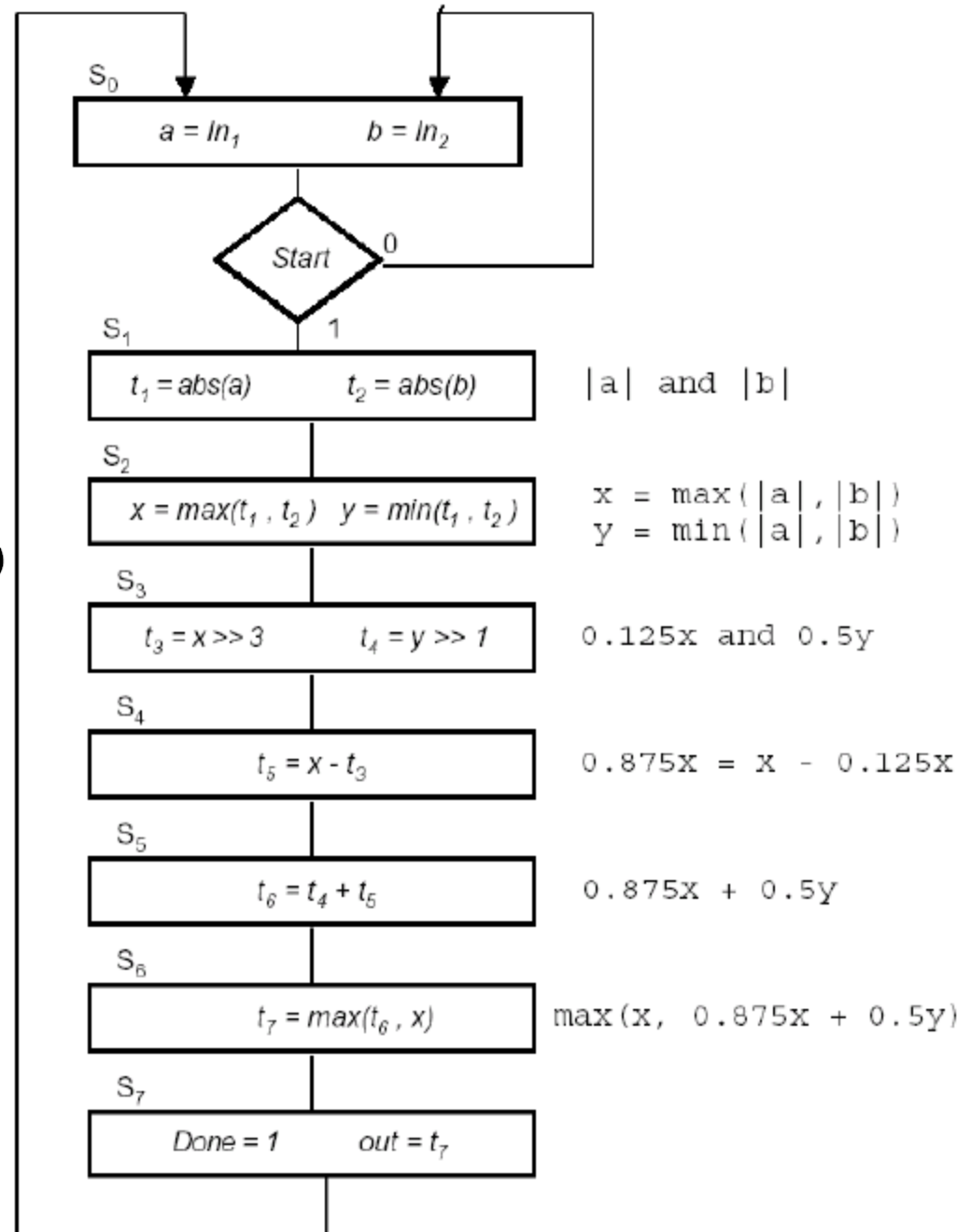
Computation units

- Min/max units

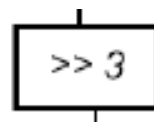
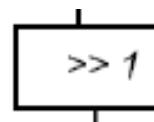
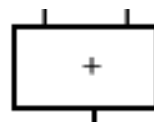
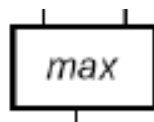
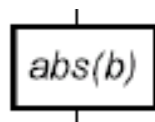
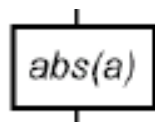
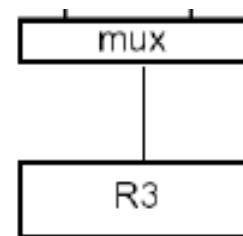
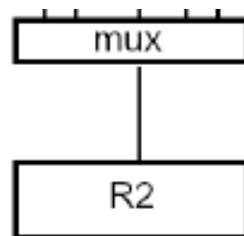
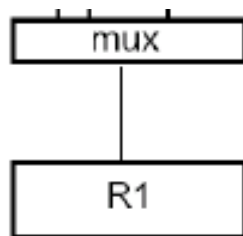


ASM

$\text{square root } (a^2 + b^2) \approx \max((0.875x + 0.5y), x)$
 $x = \max(\text{abs}(a), \text{abs}(b))$
 $y = \min(\text{abs}(a), \text{abs}(b))$



Data-path



$$\begin{aligned} R_1 &= [a, t_1, x, t_7] \\ R_2 &= [b, t_2, y, t_3, t_5, t_6] \\ R_3 &= [t_4] \end{aligned}$$

